

Decentralized and Lightweight Server-Aided Revocable Attribute-Based Encryption for Mobile Cloud Computing

Arthur Sandor Voundi Koe¹, Member, IEEE, Jianghong Wei², and Xiaofeng Chen³, Fellow, IEEE

Abstract—Mobile cloud computing (MCC) optimizes storage and computation on resource-limited mobile devices by leveraging heterogeneous networks and robust cloud servers. However, outsourcing local data to cloud servers, which may be malicious, also raises concerns about data privacy. As a prominent solution for securing outsourced data, server-aided revocable attribute-based encryption (SR-ABE) enforces fine-grained access control over encrypted data while supporting lightweight user revocation. Yet, available SR-ABE schemes struggle in practice due to the following reasons. First, they rely on a fully trusted key generation center to manage attributes and issue long-term transformation and decryption keys. Second, they require setting up a fixed number of data users, and overlook attribute revocation. Third, they incorporate time-based revocation mechanisms that assume synchrony in the underlying network, which does not always hold in practice. This paper presents a secure and practical SR-ABE scheme for MCC that supports the decentralized generation of long-term user keys while integrating state-triggered user and attribute revocation mechanisms. The proposed SR-ABE scheme accommodates unlimited data users, minimizes the sizes of key and ciphertext updates via lazy revocation, and ensures verifiable outsourced decryption. We formally prove that the proposed SR-ABE scheme is adaptively secure, and conduct extensive experiments demonstrating its practicality.

Index Terms—Attribute-based encryption, revocation, cloud, key exposure.

I. INTRODUCTION

MOBILE cloud computing (MCC) [1] aims to augment the limited computing and storage capabilities of mobile devices using cloud-based resources under a pay-per-use business model. According to recent trends [2], the amount of data created worldwide will reach 2,142 zettabytes by 2035, prompting mobile users to delegate more data to cloud servers for storage

or computation offloading. However, there are still three main challenges in the existing cloud data outsourcing scenarios. First, although encryption before outsourcing has been suggested to enhance privacy, the subsequent need to retrieve and decrypt substantial data locally is impractical. Second, only authorized individuals should have access to the data. Third, cloud servers may be malicious and perform unauthorized operations on data or provide fake or arbitrary results. Consequently, designing reliable and privacy-preserving access control techniques for encrypted data stored on cloud servers becomes essential.

Attribute-based encryption (ABE) [3] has emerged as a generalization of asymmetric encryption, enforcing one-to-many fine-grained access control on encrypted data typically hosted on untrusted yet powerful cloud servers. At its core, ABE employs an access policy based on a set of descriptive attributes. Two main types of ABE exist in the literature: key-policy (KP-ABE) [4] and ciphertext-policy (CP-ABE) [5]. In CP-ABE, the user secret key contains a set of attributes, and the ciphertext embeds an access structure, which is a mathematical formulation of the corresponding access policy. KP-ABE works as a dual of CP-ABE. Traditional ABE schemes struggle with the key escrow problem, as the central authority (CA) retains the long-term master secret key, manages the system attributes, and generates long-term user keys. Multi-authority ABE (MA-ABE) [6] was proposed to decentralize attribute management across independent attribute authorities (AAs). The challenge of eliminating the fully trusted CA and achieving a decentralized MA-ABE model (i.e., without key escrow) remained an open problem until Allison Lewko and Brent Waters [7] introduced the first truly decentralized MA-ABE scheme. Revocable ABE [8], [9] was proposed to handle user withdrawal and secret key exposure in ABE systems. As a special case of user revocation, attribute revocation [10] seeks to withdraw only a subset of user attributes [11].

Implementing revocation in ABE systems presents several critical challenges that require innovative solutions.

- 1) *Security-wise*: A revocable ABE scheme should ensure forward and backward secrecy, preventing revoked users from decrypting past or future ciphertexts, and thwart collusion attacks, preventing illegitimate participants from teaming up to elevate their decryption privileges.
- 2) *Scalability-wise*: A revocable ABE scheme should accommodate a large user base without predefined limits at setup and should minimize computational and communication demands on the data owner (DO) and

Received 5 January 2026; revised 8 April 2026; accepted 17 May 2026. Date of publication 25 May 2026; date of current version 1 September 2026. This work was supported in part by the National Cryptologic Science Fund of China under Grant 2025NCSF01011, in part by the National Natural Science Foundation of China under Grant 62432012, Grant 62572483, and Grant 62502358, in part by the Fundamental Research Funds for the Central Universities under Grant 20103247994, and in part by 111 Center under Grant B16037. (Corresponding author: Arthur Sandor Voundi Koe.)

Arthur Sandor Voundi Koe and Xiaofeng Chen are with the State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China (e-mail: sandorvoundi@xidian.edu.cn; xfchen@xidian.edu.cn).

Jianghong Wei is with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou 450001, China (e-mail: jianghong.wei.xxgc@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3696736>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3696736

data users (DUs), thus facilitating the participation of resource-constrained devices.

- 3) *Flexibility-wise*: A revocable ABE scheme should support not only user revocation but also attribute revocation, which introduces a new layer of intricacy as the same attribute can be shared by multiple DUs, and disjoint sets of attributes can be managed across independent AAs (i.e., in multi-authority settings).

Despite many revocable ABE schemes having been proposed, they still face the following fundamental issues.

- They fully trust the third-party KGC to manage the attribute universe and to compute the long-term decryption and transformation keys, as well as the key update information (KUI), resulting in key escrow and single-point-of-failure issues.
- They overlook the need for attribute revocation and require all users to be declared during system setup, limiting fine-grained access control and system scalability. In addition, a complete system reset is necessary to accommodate more users.
- They fail to operate in asynchronous networks, resulting in missed revocation updates during poor connectivity, and fail to achieve immediate revocation, thus permitting revoked users to access data throughout the active time interval, violating backward secrecy.
- They still tolerate direct revocation, which conflicts with the core principle of ABE, whereas indirect revocation preserves attribute-based settings [12].

A. Our Contributions

In this paper, we propose a decentralized SR-ABE scheme tailored for multi-authority CP-ABE scenarios that addresses the key shortcomings of prior work and supports indirect user and attribute revocation. Unlike traditional time-based revocation, we adopt state-triggered updates. In our proposal, short-term keys remain valid within a specific state and are decoupled from any assumption on time. In addition, we adopt the lazy revocation approach to statistically minimize the workload of indirect revocation. Furthermore, to address the key escrow issue in prior schemes, we decentralize the user secret key generation among DO, AAs and DUs such that none of the entities between DO and AAs can reconstruct the user secret and decrypt user messages on their own. We implement our construction in Charm, a programming framework for rapid prototyping of cryptographic primitives, and provide the ready-to-use source code.

We summarize the main contributions of this work as follows:

- We enforce flexible user and attribute indirect revocation, uphold a large polynomial number of revoked and unrevoked users without system reset, and statistically reduce the key update overhead.
- We decentralize the user secret key generation between stateful DO and AAs, delegate the key and ciphertext revocation update processes to a platform-agnostic TEE-capable cloud server with transparent enclave execution, as well as achieve verifiable outsourced decryption within multi-authority CP-ABE settings.

- We introduce a novel security model that captures the threat of transformation key exposure attacks in state-triggered revocation settings via leakage and corruption.

B. Related Work

The current literature distinguishes two broad revocation mechanisms [12]: direct revocation [13], [14], [15] and indirect revocation [16], [17]. In direct revocation, DO embeds a revocation list within the ciphertext to achieve immediate revocation. In contrast, indirect revocation allows the non-revoked users to enjoy secret key updates through the dissemination of key update information (KUI), which may occur over public or private channels. In their compelling approach, Li et al. [18] leveraged a user revocation list (UL) for efficient direct revocation, where UL lists authorized users. A key generation center (KGC) that is fully trusted routinely signs a new state with UL at each time interval's start. What is interesting is that Li et al. [18] exploit a trusted execution environment (TEE) at the cloud service provider (CSP) level to construct their threat model and prove the resilience of their design against key exposure attacks. Kevin Fu's insightful study [19] tackled the issue of frequent (indirect) revocation overload by introducing lazy revocation, a method that postpones key and ciphertext updates until subsequent data access, such as read or write operations [20], thus statistically minimizing the computational load of the update process [21]. Yang et al. [22] employed a semi-trusted cloud server to conduct attribute revocation via attribute version control (AVC). AVC assigns a version number to each attribute, updating it upon revocation. Indirect revocation can optionally allow ciphertexts to be recomputed to embed the latest update information, therefore preventing revoked users from data access.

The concept of Server-aided revocable ABE (SR-ABE) [23], [24], [25] aims to transfer extensive user operations, such as revocation and ciphertext decryption, to an untrusted CSP. Within such a framework, the KGC retains the long-term master secret key, issues long-term secret keys to DUs and grants long-term transformation keys to the server for ciphertext transformation. In addition, DUs can delegate decryption capabilities by issuing short-term decryption keys that remain valid for a specified period of time. In SR-ABE, regular updates by the KGC through KUI prompt changes in long-term transformation or secret keys [26], resulting in the creation of short-term transformation or decryption keys. Decryption key exposure (DKE) attacks [27] occur when an adversary gains access to a short-term decryption key, due to leak or theft, alongside the KUI through a public channel, enabling such adversary to reconstruct the long-term secret key and generate subsequent short-term decryption keys. Xu et al. [28] suggested that the exposure of the transformation key can be overlooked if the KUI is public, as the transformation key can be derived by anyone. The notion of secure SR-ABE [29] aims to guarantee that a dishonest CSP is unable to carry out the final decryption stage even when colluding with revoked users.

Recent studies have sought to enhance revocation mechanisms in ABE by approaching the problem from various perspectives. Guo et al. [30] implemented user revocation by employing a binary user tree of depth d under fully trusted

authority, dynamic user expansion, and small attribute universe settings. Moreover, the ciphertext update process relies on tight assumptions on timestamp thresholds. Yin et al.[31] proposed a traceable and revocable CP-ABE scheme that relies on a fully trusted authority and avoids exposing the revocation list, thus supporting secure verification of identity membership via secure aggregation functions. Fei Meng and Leixiao Cheng [32] introduced a white-box traceable, server-aided revocable CP-ABE scheme built on a fully trusted KGC, a small attribute universe, and composite-order groups. Their design supports time-based revocation via a binary user tree and has the KGC send long-term transformation keys directly to the cloud server. Li et al.[33] proposed a white-box traceable and (direct) revocable CP-ABE scheme resistant to fake policy attacks. Their design assumes fully trusted KGC and policy checker components, uses a binary user tree, and accommodates a small attribute universe. Sun et al. [34] introduced a privacy-preserving inner-product CP-ABE encryption scheme for cloud-edge environments that offers fully hidden policies, outsourced decryption, indirect revocation, and trapdoor-based ciphertext search. Their design assumes a small attribute universe, time-based revocation, and a fully trusted authority that directly distributes secret but updatable transformation keys to edge devices.

Although these schemes are relatively new, they nonetheless retain the core disadvantages of traditional revocable ABE constructions. These issues are typical in outsourced storage environments [35], such as encrypted cloud email services [36] and attribute-based signature mechanisms [37], as well as in outsourced computation frameworks [38].

C. Outline

We structure the remainder of our paper as follows. Section II delves into the preliminaries. Section III details the system and security models of our framework, then gives its mathematical specifications. Section IV analyzes the security of our construction. Section V evaluates its theoretical and empirical performance and presents our findings. Finally, Section VI concludes our paper.

II. PRELIMINARIES

This section reviews key preliminaries necessary to comprehend our paper, including a formal definition of the baseline CP-ABE scheme [5]. In addition, Table I below highlights the main abbreviations and symbols used throughout this paper to ease their reference.

A. Bilinear Maps

Consider GroupGen as a probabilistic polynomial-time (PPT) algorithm that, upon receiving a security parameter 1^λ , outputs a group description $\mathbb{G} := (p, \mathbb{G}_0, \mathbb{G}_T, e, g)$, where p is a prime of $\Theta(\lambda)$ bits, $\mathbb{G}_0$ is a multiplicative cyclic group of large prime order p , and g is a generator of $\mathbb{G}_0$. We note that the group description might contain additional parameters. We set e as a bilinear map such that $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_T$ is the target group and

TABLE I
SUMMARY OF ABBREVIATIONS AND SYMBOLS

Abbreviation or Symbol	Definition
DO	Data Owner
DU	Data User
AA	Attribute Authority
$\text{root}(\mathcal{T})$	Root node of tree $\mathcal{T}$
$\text{att}(y)$	Attribute value of leaf node y in $\mathcal{T}$
$\mathbb{S}_x$	Set of child nodes of node x in $\mathcal{T}$
$\text{index}(z z \in \mathbb{S}_x)$	Index of each child node in $\mathbb{S}_x$
CRS	Common reference string
gid	Global user identity
UL	User revocation list
MHT _{UL}	Merkle hash tree constructed from UL
AL	Attribute revocation list of a given AA
TEE-CS	Cloud server with TEE capabilities
$(\text{pub}_{\text{DO}}, \text{priv}_{\text{DO}})$	Public and private key pair of DO
$(\text{pub}_{\text{DU}}, \text{priv}_{\text{DU}})$	Public and private key pair of DU
$(\text{pub}_{\text{AA}}, \text{priv}_{\text{AA}})$	Public and private key pair of AA
DO_{key}	DU partial secret key issued by DO
σ_{DO}	DO private state for user revocation
umrk	Master recomputation key by DO
$\text{crk}_{\text{DO}}^{\text{urev}}$	Ciphertext recomputation key by DO
$(\text{sk}_{\text{DO}}, \text{hk}_{\text{DO}(1)}, \dots, \text{hk}_{\text{DO}(3)})$	Ring of keys in DO_{key}
st_{DO}	DO's public state index
st_i^{DO}	i^{th} tuple in DO's public state index st_{DO}
$\text{B}_{\sigma_{\text{DO}}}$	DO's private state index
$\text{AA}_{\text{key}}^{\text{(PRF(gid), S}_{\text{DU}})}}$	DU partial secret key issued by AA
σ_{AA}	AA private state
amrk	Master recomputation key by AA
st_{AA}	AA's public state index
st_i^{AA}	i^{th} tuple in AA's public state index st_{AA}
$\text{B}_{\sigma_{\text{AA}}}$	AA's private state index
$\text{TK}_{\text{DU}}^{\text{SDU}}$	DU transformation key
$\text{st}_{\text{AA}}^{\text{AA}}[\text{counter}_{\text{AA}}]$	tuple indexed by $\text{counter}_{\text{AA}}$ in st_{AA}
$\text{st}_{\text{DO}}^{\text{DO}}[\text{counter}_{\text{DO}}]$	tuple indexed by $\text{counter}_{\text{DO}}$ in st_{DO}
attr_{set}	Attribute table with version numbers
AT	Table of AAs
SL	Attribute table with version numbers
UT	Table of users
RT_{DO}	DO's public state index
RT_{AA}	AA's public state index
TC	Transformed ciphertext
I	Term precomputed by TEE-CS for DU

$e(g, g)$ stands as the generator of $\mathbb{G}_T$. The bilinear map e has the following properties:

- *Bilinearity*: $\forall u, v \in \mathbb{G}_0$ and $a, b \in \mathbb{Z}_p$, $e(u^a, v^b) = (e(u, v))^{a,b}$
- *Non-degeneracy*: $e(g, g) \neq 1_{\mathbb{G}_T}$

$(\mathbb{G}_0, \mathbb{G}_T)$ is a bilinear group if the group operation in $\mathbb{G}_0$ as well as the bilinear map $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_T$ are efficiently computable. Notice that the map e is symmetric since the bilinear operation $e(\cdot, \cdot)$ maps pairs of elements from the same source group $\mathbb{G}_0$ to the target group.

B. Ciphertext-Policy ABE

Drawing inspiration from Bethencourt et al.'s notable CP-ABE model [5], we develop an innovative decentralized CP-ABE framework that supports user and attribute revocation in multi-authority settings and accommodates large attribute universe scenarios. We give a systemic definition of the work of Bethencourt et al.[5] that relies on four main algorithms: Setup, Keygen, Encrypt, and Decrypt.

- *Setup*(1^λ) $\rightarrow (PP, MSK)$: The setup algorithm takes as input the security parameter 1^λ . It outputs the public parameters PP and the long-term master secret key MSK .

- **Setup**(1^λ): Run $\text{GroupGen}(\lambda)$ to obtain the group description $\mathbb{G} := (p, \mathbb{G}_0, \mathbb{G}_T, e, g)$. Pick $\alpha, \beta \xleftarrow{\$} \mathbb{Z}_p^* \times \mathbb{Z}_p^*$ and a hash function $\mathcal{H}_1 : \{0, 1\}^* \rightarrow \mathbb{G}_0$. Compute the master secret key as $\text{MSK} := (\beta, g^\alpha)$, as well as two values $h := g^\beta$ and $f := g^{1/\beta}$. Compute the public parameters as $\text{PP} := (\mathbb{G}, h, f, \mathcal{H}_1, e(g, g)^\alpha)$.
- **Keygen**($\text{PP}, \text{MSK}, \mathbb{S} := \{\text{attr}_1, \text{attr}_2, \text{attr}_3, \dots, \text{attr}_n\}$): Pick $r \xleftarrow{\$} \mathbb{Z}_p$. Compute $D := g^{(\alpha+r)/\beta}$. For each attribute $j \in \mathbb{S}$, pick $r_j \xleftarrow{\$} \mathbb{Z}_p^*$. Compute $D_j := g^r \cdot \mathcal{H}_1(j)^{r_j}$, and $T_j := g^{r_j}$. Publish $\text{SK} := (D, \{(D_j, T_j)\}_{j \in \mathbb{S}})$.
- **Encrypt**($\text{PP}, M, \mathcal{T}$): Pick $s \xleftarrow{\$} \mathbb{Z}_p^*$. Compute $\text{CS} := M \cdot e(g, g)^{\alpha \cdot s}$, and $C := g^{\beta \cdot s}$. Define a polynomial $q(x)$ for every node $x \in \mathcal{T}$ in top-down manner starting from $\text{root}(\mathcal{T})$. For every leaf node $y \in \mathcal{T}$, compute $C_y := g^{q_y(0)}$, and $T_y := \mathcal{H}_1(\text{att}(y))^{q_y(0)}$, where $\text{att}(y)$ returns the attribute value associated with y . Publish $\text{CT} := (\mathcal{T}, \text{CS}, C, \{(C_y, T_y)\}_{y \in \mathcal{T}})$.
- **Decrypt**($\text{PP}, \text{CT}, \text{SK}$): Define the following recursion. For each leaf node $y \in \mathcal{T}$ such that $\text{att}(y)$ corresponds to $j \in \mathbb{S}$, compute $F_y := e(D_j, C_y) / e(T_j, T_y) = e(g, g)^{r \cdot q_y(0)}$. For each child z of x , set $\mathbb{S}'_x := \{(\text{index}(z) | z \in \mathbb{S}_x)\}$ and compute $F_x := \prod_{z \in \mathbb{S}_x} F_z^{\Delta_{i, \mathbb{S}'_x}(0)} = e(g, g)^{r \cdot q_x(i) \cdot \Delta_{i, \mathbb{S}'_x}(0)} = e(g, g)^{r \cdot q_x(0)}$. Compute $(\text{CS}) \cdot (F_{\text{root}(\mathcal{T})}) / e(\text{CS}, D)$ and output the result. If $\mathbb{S}$ satisfies $\mathcal{T}$, then $F_{\text{root}(\mathcal{T})} := e(g, g)^{r \cdot s}$.

Fig. 1. Overview of the baseline CP-ABE scheme.

- **Keygen**($\text{PP}, \text{MSK}, \mathbb{S}$) $\rightarrow$ SK . The key generation algorithm takes as inputs the public parameters PP , the master secret key MSK and a set of user attributes $\mathbb{S}$. It outputs the complete user secret key SK .
- **Encrypt**($\text{PP}, M, \mathcal{T}$): The encryption algorithm takes as inputs the public parameters PP , the payload M , and the access structure $\mathcal{T}$ over the universe of attributes $\mathbb{U}$. It outputs the ciphertext CT , which includes $\mathcal{T}$.
- **Decrypt**($\text{PP}, \text{CT}, \text{SK}$): The decryption algorithm takes as inputs the public parameters PP , the ciphertext CT , and the private key SK . If the set of attributes $\mathbb{S}$ in SK satisfies $\mathcal{T}$ in CT , the decryption algorithm outputs M . Otherwise, the decryption algorithm stops and returns $\perp$, which denotes a random string with overwhelming probability or no output at all.

We describe an overview of the baseline CP-ABE of Bethencourt et. al [5] in Fig. 1.

C. Access Structures

Let $\mathbb{U}$ be the universe of attributes. We define an access structure on $\mathbb{U}$ as a collection $\mathbb{C} \subseteq 2^\mathbb{U} \setminus \emptyset$ that contains non-empty sets of attributes. All sets in $\mathbb{C}$ are referred to as *authorized sets*. All sets not in $\mathbb{C}$ are referred to as *unauthorized sets*. In addition, an access structure qualifies as monotone if $\forall D, E \in 2^\mathbb{U}$, if $D \in \mathbb{C}$ and $D \subseteq E$, then $E \in \mathbb{C}$. In other words, any superset E that includes the authorized set D must also satisfy the access structure $\mathbb{C}$.

D. Decisional Bilinear Diffie-Hellman Assumption

Let $(\mathbb{G}_0, \mathbb{G}_T)$ be a bilinear group where g generates $\mathbb{G}_0$, and $e(g, g)$ generates $\mathbb{G}_T$. The decisional bilinear Diffie-Hellman (DBDH) hardness assumption in symmetric settings states the following: given $A := g^a$, $B := g^b$, $C := g^c$, and $e(g, g)^{a \cdot b \cdot c}$ as inputs, it is computationally infeasible for any probabilistic polynomial time (PPT) adversary to distinguish $e(g, g)^{a \cdot b \cdot c}$ from

$e(g, g)^z$ with more than a negligible advantage, where a, b, c, z are chosen randomly from $\mathbb{Z}_p$.

III. OUR FRAMEWORK

A. System Model

We integrate a layered computational framework that comprises a foundational public-key layer beneath an ABE layer. The public-key layer manages the computations related to the generation, distribution, and validation of asymmetric cryptographic primitives. The ABE layer operates atop the public-key layer and handles computations related to CP-ABE functionalities. We assume a one-time trusted sampling of a structured common reference string (CRS) that we restrict to public elliptic curve domain parameters. We let DO generate the setup parameters at the ABE layer but prevent it from decrypting user messages. We split the generation of user keys at the ABE layer between DO and each requested AA. Each DU assembles its complete user key at the ABE layer by merging key components from DO and AAs, then stores it locally as a key ring.

This work adopts a TEE-capable, platform-agnostic cloud server (TEE-CS) that operates under the transparent enclave execution model [39]. This abstraction captures isolated execution environments that ensure execution integrity and provide attestation, while presuming that the enclave's entire internal state is fully transparent (i.e., no confidentiality), thereby granting the adversary unrestricted information leakage. In addition, different providers could adopt a variety of techniques to realize a TEE that supports a transparent enclave execution model, yet our approach would still be applicable to each of them. TEE-CS exhibits four main modules as in Li et al. [18], though with a distinct workflow.

In our scheme, key and ciphertext update information are encapsulated within (digitally signed) public states sent to TEE-CS to enforce revocation updates. For user revocation, DO updates a global user revocation list (UL) containing the gid of each

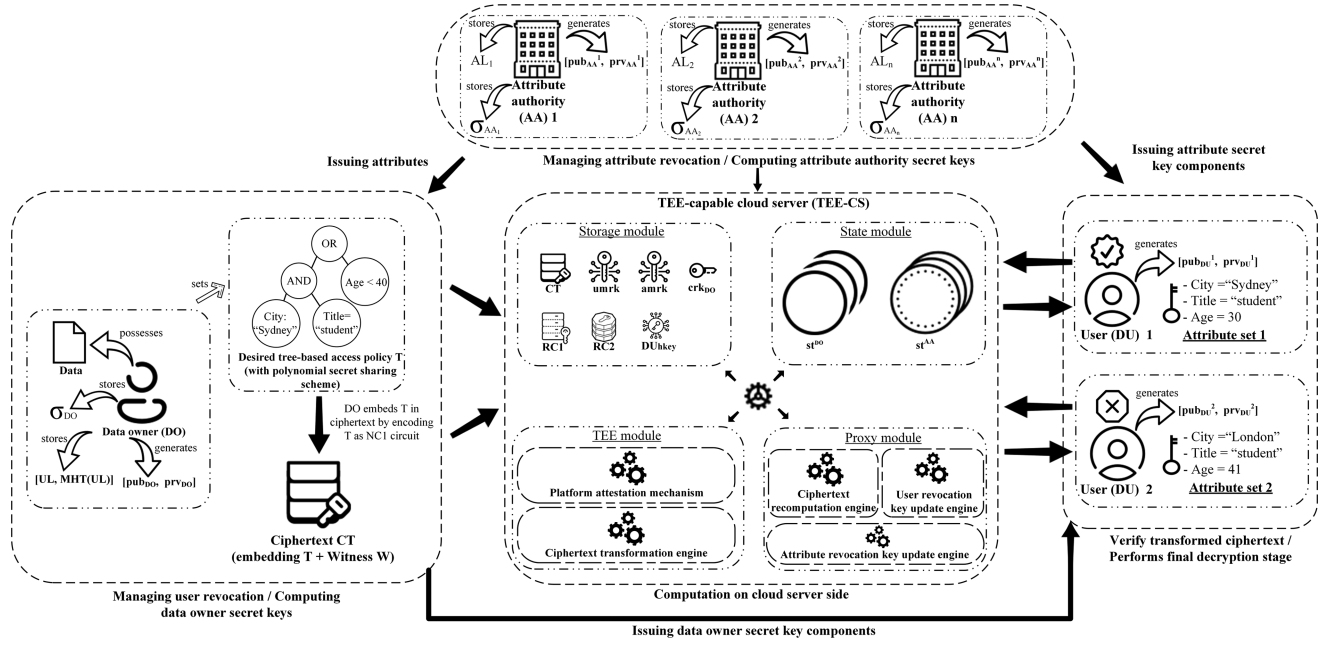


Fig. 2. System model of our server-aided revocable ABE framework

revoked user, and this UL is included in the public state sent to TEE-CS. To mitigate the cost of loading the entire UL for each revocation, the signed Merkle hash tree (sMHT) method is employed, with MHT_{UL} representing UL in the form of a Merkle hash tree. Regarding attribute revocation, each AA generates a table $attr_set$ that lists all attributes under its control together with their associated version numbers, as well as an attribute revocation list (AL) that specifies the attributes to be revoked and the updated version numbers of these attributes that must be enforced. Leveraging the AVC mechanism, each AA increments the versions of revoked attributes under its management by one. Subsequently, AA generates a public state that includes AL along with a digital signature over AL, then dispatches this public state to TEE-CS. To enjoy ciphertext transformation and key updating, DUs outsource their transformation keys to TEE-CS, which also handles ciphertext recomputation to update ciphertexts. DO and AAs keep private local states to help create public states for TEE-CS, which are used to update the transformation keys of non-revoked users.

In summary, our framework consists of four main entities: DO, DU, TEE-CS, and AA. Fig. 2 illustrates our system model, and the roles of each component are described below.

- **DO:** DO generates a public and private key pair $(pub_{DO}, priv_{DO})$ at the public-key layer. It computes the system's public parameters PP and the master secret key MSK at the ABE layer. It encrypts any message M from the message space under a threshold tree-based access structure T , encoded as NC1 circuit. In addition, DO embeds a witness W within the resulting ciphertext CT, then uploads CT to TEE-CS for access control implementation. DO computes a partial user key DO_{key} for any given DU, at the ABE layer. DO_{key} contains a secret key sk_{DO} as well as

three helper keys $hk_{DO(1)}$, $hk_{DO(2)}$, and $hk_{DO(3)}$, such that $DO_{key} = (sk_{DO}, hk_{DO(1)}, hk_{DO(2)}, hk_{DO(3)})$.

To enforce user revocation, DO performs several tasks. First, it generates a new tuple σ_i^{DO} within the private state index σ^{DO} , where i acts as an internal counter, such that σ_i^{DO} holds the update information. Second, it computes a ciphertext recomputation key crk_{DO}^{DO} and subsequently forms a master recomputation key $umrk$, both derived from σ_i^{DO} . We note that $umrk$ will be used to update $hk_{DO(2)}$ and $hk_{DO(3)}$, while crk_{DO}^{DO} will be used to update the targeted ciphertext(s). Third, DO generates a new tuple st_i^{DO} inside the public state index st^{DO} , where i serves as an internal counter (to keep track of user revocation states). st_i^{DO} comprises both MHT_{UL} and a signature on the root of MHT_{UL} , which we express as Sig_{UL} . We have $st_i^{DO} = (MHT_{UL}^{(i)}, Sig_{UL}^{(i)})$. Fourth, DO transmits $(st_i^{DO}, crk_{DO}^{DO}, umrk)$ to TEE-CS.

We note that the number of tuples in σ^{DO} and st^{DO} , respectively, depends on DO's storage and processing capacities, as well as on the valuation DO assigns to keeping historical tuples.

- **AA:** AA generates a pair of public and secret keys $(pub_{AA}, priv_{AA})$ at the public-key layer. It oversees a unique attribute set S_{AA} , and can access any attribute in S_{AA} or its latest version as required. AA computes the attribute authority secret key $AA_{key}^{(PRF(gid), S_{DU})}$ using $priv_{AA}$ and DU's gid as inputs to a specific PRF of public knowledge, together with the designated subset of attributes S_{DU} (under management) requested by DU.

To enforce attribute revocation, AA undertakes a sequence of actions. AA generates a fresh tuple σ_i^{AA} within the private state index σ^{AA} , where i acts as an internal counter and σ_i^{AA} formulates the update information. Next, AA

generates a new tuple st_i^{AA} inside the public state index st^{AA} , where i serves as an internal counter (to keep track of attribute revocation states). st_i^{AA} comprises both AL and a signature on AL, expressed as Sig_{AL} . We illustrate st_i^{AA} as $st_i^{AA} = (AL^{(i)}, Sig_{AL}^{(i)})$. Subsequently based on st_i^{AA} , AA calculates a distinct master recomputation key $amrk_j$ for each attribute in AL such that $j \in [1, \dots, \text{len}(AL)]$, where the function $\text{len}()$ returns the length of any iterable argument. For simplicity, we let AA generate a unified master recomputation key $amrk$ per revocation state σ_i^{AA} , which covers all the attributes in the target AL. Finally, AA uploads the tuple $(st_i^{AA}, amrk)$ to TEE-CS.

We note that since the objective is to update the attribute-related key and ciphertext components by introducing new exponents, using either a distinct novel $amrk_j$ for each attribute or a single fresh $amrk$ shared across all attributes in the target AL yields the same outcome. However, generating a single $amrk$ value is more efficient than computing several distinct $amrk_j$ values, particularly when the size of the target AL becomes large. As a related example, the interesting scheme of Fei Meng and Leixiao Cheng [32] optimizes the generation of long-term transformation keys proposed in their previous work [40] by assigning one shared random element to all nodes associated with a user's identity in the binary user tree, instead of giving each node its own independent random element.

The number of tuples in st^{AA} and σ^{AA} , respectively, is governed by AA's storage and processing resources, as well as by the degree of importance AA places on preserving historical tuples.

- **DU:** DU generates a public and private key pair $(pub_{DU}, priv_{DU})$ at the public-key layer. It receives DO_{key} from DO and k instances of $AA_{key}^{(PRF(gid), S_{DU})}$ from k designated AAs. DU establishes its complete user key DU_{key} at the ABE layer as $DU_{key} = (DO_{key}, TK_{DU}^{S_{DU}} = (AA_{key}^{(PRF(gid), S_{DU})})_{(1, \dots, k)})$, where $TK_{DU}^{S_{DU}}$ stands as DU's transformation key. DU establishes a user helper key $DU_{hkey} = (hk_{DO(1)}, hk_{DO(2)}, hk_{DO(3)}, TK_{DU}^{S_{DU}})$ which it delegates to TEE-CS. DU initiates the ciphertext transformation operation within TEE-CS, and executes the final decryption stage on-premises. We characterize a legitimate DU as one whose S_{DU} satisfies $\mathcal{T}$, while an illegitimate DU is either a revoked DU or one whose S_{DU} fails to meet $\mathcal{T}$ (i.e., unauthorized DU).
- **TEE-CS:** TEE-CS exhibits adequate computational, storage, and energy supply capabilities. Its architecture integrates four distinct modules: the storage, state, proxy, and TEE modules.
 - 1) **Storage module:** the storage module offers data storage services to all participants, enabling data queries from both external participants and other TEE-CS modules via a generic application programming interface (API). The storage module can also push requests and data to other TEE-CS modules.
 - 2) **State module:** the state module archives public states provided by DO and AAs. It offers an API to manage

those states and fetch the most recent one for DO and any AA, respectively. Upon availability of a new valid state, the state module prompts the storage module to construct and forward two types of request to the proxy module: an immediate and one-time ciphertext recomputation request; a lazy and one-time user helper key (i.e., DU_{hkey}) update request per user.

- 3) **Proxy module:** The proxy module serves as the core computational element of TEE-CS, tasked with ciphertext recomputation and user helper key updates.

- ★ **ciphertext recomputation for user revocation:** The proxy module obtains crk_{urev}^{DO} and CT from the storage module's request, then recomputes CT. We label the recomputed ciphertext as RC1. The proxy module transmits RC1 to the storage module that overwrites the previous CT value.

- ★ **ciphertext recomputation for attribute revocation:** The proxy module gets $amrk$ and CT from the storage module's request, then recomputes CT. We label the recomputed ciphertext as RC2. The proxy module transmits RC2 to the storage module that overwrites the former CT value.

- ★ **user helper key update for user revocation:** The proxy module fetches the most recent st_i^{DO} from the state module, and acquires DU_{hkey} (which contains $hk_{DO(2)}, hk_{DO(3)}$) and $umrk$ from the storage module's request. It verifies the authenticity of $MHT_{UL}^{(i)}$ using $Sig_{UL}^{(i)}$ and pub_{DO} , then calculates the Merkle proof from $MHT_{UL}^{(i)}$ using the hash digest of the current requesting DU's gid , determining its revocation status. If unrevoked, the proxy module updates $hk_{DO(2)}$ and $hk_{DO(3)}$ with $umrk$. The proxy module returns the updated version of DU_{hkey} to the storage module that overwrites the previous value. Given a revoked DU, the proxy module halts and outputs $\perp$.

- ★ **user helper key update for attribute revocation:** The proxy module obtains the latest st_i^{AA} and st_i^{DO} from the state module, as well as DU_{hkey} together with $amrk$ from the storage module's request. After assessing that the requesting DU is unrevoked, the proxy module updates DU_{hkey} with $amrk$, and returns the updated value of DU_{hkey} to the storage module that overwrites the old value. If DU is revoked, the proxy module stops and outputs $\perp$.

- 4) **TEE module:** The TEE module conducts ciphertext transformation upon DU's request and proceeds as follows. First, it retrieves CT in the absence of revocation events, or alternatively, acquires RC1 or RC2 from DU's request. Second, it extracts DU_{hkey} from DU's request. If DU is legitimate, the TEE module produces a transformed ciphertext TC, which is sent to the requesting DU along with a signature attestation sig_{TC} on TC and

- **Setup(CRS):** pick $a, b, \alpha, \beta \xleftarrow{\$} \mathbb{Z}_p^* \times \mathbb{Z}_p^* \times \mathbb{Z}_p^* \times \mathbb{Z}_p^*$. Compute $\text{priv}_{\text{DO}} := a \cdot b$, and $\text{pub}_{\text{DO}} := g^{a \cdot b}$. Compute $\text{MSK} := (\beta, g^\alpha)$ and $\text{PP} := (h = g^\beta, f = g^{1/\beta}, e(g, g)^\alpha)$.
- **SetupAA(CRS, aa_{id}, S_{AA}):** pick $(a'', b'') \xleftarrow{\$} \mathbb{Z}_p^* \times \mathbb{Z}_p^*$. Compute $\text{priv}_{\text{AA}} := a'' \cdot b''$, and $\text{pub}_{\text{AA}} := g^{a'' \cdot b''}$. For every attribute $\text{attr} \in \text{S}_{\text{AA}}$, initialize the version of attr to 0 and define $\text{attr_set}[\text{attr}] := (\text{attr}, 0)$.
- **RegUser(CRS, gid, S):** pick $(a', b') \xleftarrow{\$} \mathbb{Z}_p^* \times \mathbb{Z}_p^*$. Compute $\text{priv}_{\text{DU}} := a' \cdot b'$, and $\text{pub}_{\text{DU}} := g^{a' \cdot b'}$.
- **Keygen:**
 - **Keygen1(CRS, MSK, gid, pub_{DU}):** pick $r \xleftarrow{\$} \mathbb{Z}_p^*$. Compute $\epsilon := \mathcal{H}_0(\text{gid})$. Besides, compute $\text{sk}_{\text{DO}} := g^{(\alpha + r \cdot a' \cdot b') / \beta(1 + \epsilon)}$, $\text{hk}_{\text{DO}(1)} := g^{(\epsilon + r \cdot a' \cdot b') / (1 + \epsilon)}$, $\text{hk}_{\text{DO}(2)} := g^{\epsilon / \beta(1 + \epsilon)}$, and initialize $\text{hk}_{\text{DO}(3)} := \perp$. Output $\text{DO}_{\text{key}} := (\text{sk}_{\text{DO}}, \text{hk}_{\text{DO}(1)}, \text{hk}_{\text{DO}(2)}, \text{hk}_{\text{DO}(3)})$.
 - **Keygen2(CRS, $\epsilon := \mathcal{H}_0(\text{gid})$, priv_{AA} , $\text{S}_{\text{DU}} := \{\text{attr}_1, \text{attr}_2, \text{attr}_3, \dots, \text{attr}_n\}$):** Set $\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})} := \{\}$. For each attribute $\text{attr} \in \text{S}_{\text{DU}}$ indexed by j , pick $r_j \xleftarrow{\$} \mathbb{Z}_p^*$. Compute $D_j := g^{(\mathcal{H}_0(\text{attr}) \cdot r_j / (a'' \cdot b'' + \epsilon))} \cdot g^{a'' \cdot b'' \cdot \mathcal{H}_0(\text{attr})} \cdot g^{\mathcal{H}_0(\text{attr}) \cdot r_j}$, and $T_j := g^{a'' \cdot b'' \cdot (r_j / (a'' \cdot b'' + \epsilon))} \cdot g^{r_j}$. Set $\text{sk}_j := (D_j, T_j, \text{attr})$ and compute $\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})} := \text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})} \cup \{\text{sk}_j\}$. Output $\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})}$.
 - **Keygen3(CRS, $\{\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})}\}_{1, \dots, k}$):** Initialize $\text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}$ as an empty dictionary. For each $\text{aa_key} \in \{\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{H}_0(\text{gid})), \text{S}_{\text{DU}})}\}_{1, \dots, k}$, define $\text{attr} := \text{aa_key}[3]$, set $\text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}[\text{attr}] := \text{aa_key}$, and compute $\text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}[\text{S}_{\text{attr}}] := \text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}[\text{S}_{\text{attr}}] \cup \text{attr}$. Output $\text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}$.
 - **Keygen4(CRS, DO_{key} , $\text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}$):** Compute $\text{DU}_{\text{key}} := \text{DO}_{\text{key}} \cup \text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}}$, and $\text{DU}_{\text{hkey}} := (\text{hk}_{\text{DO}(1)}, \text{hk}_{\text{DO}(2)}, \text{hk}_{\text{DO}(3)}, \text{TK}_{\text{DU}}^{\text{S}_{\text{DU}}})$. Output $(\text{DU}_{\text{key}}, \text{DU}_{\text{hkey}})$.
- **Encrypt(PP, M, T, priv_{DO}):** Pick $s \xleftarrow{\$} \mathbb{Z}_p^*$. Compute $\text{CS} := M \cdot e(g, g)^{\alpha \cdot s}$, $C := g^{\beta \cdot s}$, $W := \mathcal{H}_1(\mathcal{H}_2(M))$, and $\text{sig}_W := \prod_{2_{\text{sign}}}(\text{priv}_{\text{DO}}, W)$. Define a polynomial $q(x)$ for every node $x \in \mathcal{T}$ in top-down manner starting from $\text{root}(\mathcal{T})$. For every leaf node $y \in \mathcal{T}$, compute $D_y := g^{q_y(0)}$, $T_y := g^{\mathcal{H}_0(\text{att}(y))^{q_y(0)}}$, where $\text{att}(y)$ returns the attribute value associated with y , and $S_y := \text{index}(y)$, where $\text{index}(y)$ returns the index of node y within $\mathcal{T}$. Publish $\text{CT} := (\mathcal{T}, \text{CS}, C, W, \text{sig}_W, \{(D_y, T_y, S_y)\}_{y \in \mathcal{T}})$.
- **Revoke:**
 - **Revoke1(CRS, MSK, σ^{DO} , st^{DO} , UL, priv_{DO}):** Compute $\text{counter}_{\text{DO}} := \text{counter}_{\text{DO}} + 1$, where $\text{counter}_{\text{DO}}$ starts at 0. If user revocation occurs for the first time, set $\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][\lambda'] := 0$ and pick $\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][z'] \xleftarrow{\$} \mathbb{Z}_p^*$. Otherwise, compute $\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][\lambda'] := \sigma^{\text{DO}}[\text{counter}_{\text{DO}} - 1][z']$ and pick $\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][z'] \xleftarrow{\$} \mathbb{Z}_p^*$. Using UL, which contains the set of user gids to be revoked, compute $\text{MHT}_{\text{UL}} := \text{MerkleTree}(\text{UL})$, then set $\text{st}^{\text{DO}}[\text{counter}_{\text{DO}}] := (\text{MHT}_{\text{UL}}, \text{Sig}_{\text{UL}} = \prod_{2_{\text{sign}}}(\text{priv}_{\text{DO}}, \text{root}(\text{MHT}_{\text{UL}})))$. Next, compute $\text{crk}_{\text{urev}}^{\text{DO}} := g^{(\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][z'] - \sigma^{\text{DO}}[\text{counter}_{\text{DO}}][\lambda']) \cdot a \cdot b \cdot s}$ and $\text{umrk} := g^{((\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][z']) \cdot a \cdot b / \beta) / g^{((\sigma^{\text{DO}}[\text{counter}_{\text{DO}}][\lambda']) \cdot a \cdot b / \beta)}}$. For subsequent operations, define $\text{st}_i^{\text{DO}} := \text{st}^{\text{DO}}[\text{counter}_{\text{DO}}]$.
 - **Revoke2(σ^{AA} , attr_list , priv_{AA} , st^{AA} , attr_set):** Compute $\text{counter}_{\text{AA}} := \text{counter}_{\text{AA}} + 1$, where $\text{counter}_{\text{AA}}$ starts at 0. If attribute revocation occurs for the first time, set $\sigma^{\text{AA}}[\text{counter}_{\text{AA}}][\eta'] := 1$ and pick $\sigma^{\text{AA}}[\text{counter}_{\text{AA}}][k'] \xleftarrow{\$} \mathbb{Z}_p^*$. Otherwise, compute $\sigma^{\text{AA}}[\text{counter}_{\text{AA}}][\eta'] := \sigma^{\text{AA}}[\text{counter}_{\text{AA}} - 1][k']$ and pick $\sigma^{\text{AA}}[\text{counter}_{\text{AA}}][k'] \xleftarrow{\$} \mathbb{Z}_p^*$. For every attribute attr in attr_list , provided that attr is managed by the current AA, compute $\text{attr_set}[\text{attr}][2] := \text{attr_set}[\text{attr}][2] + 1$ (i.e., increment the version of attr by one), and set $\text{st}^{\text{AA}}[\text{counter}_{\text{AA}}][\text{AL}'][\text{attr}] := (\text{attr_set}[\text{attr}][1], \text{attr_set}[\text{attr}][2])$. Define $\text{st}^{\text{AA}}[\text{counter}_{\text{AA}}] := (\text{st}^{\text{AA}}[\text{counter}_{\text{AA}}][\text{AL}'], \text{Sig}_{\text{AL}} = \prod_{2_{\text{sign}}}(\text{priv}_{\text{AA}}, \text{st}^{\text{AA}}[\text{counter}_{\text{AA}}][\text{AL}']))$, and compute $\text{amrk} := \sigma^{\text{AA}}[\text{counter}_{\text{AA}}][k'] \cdot \sigma^{\text{AA}}[\text{counter}_{\text{AA}}][\eta']$. For subsequent operations, set $\text{st}_i^{\text{AA}} := \text{st}^{\text{AA}}[\text{counter}_{\text{AA}}]$.

Fig. 3. Our decentralized and lightweight SR-ABE scheme (part I).

some precomputed term I that DU will use for local decryption. If DU 's attributes fail to satisfy $\mathcal{T}$, the TEE module outputs $\perp$.

We note that sig_{TC} provides non-repudiation guarantees against TEE-CS. Throughout the rest of this paper, we omit sig_{TC} , except within the Transform operation in Fig. 4, where its mathematical formulation is established.

B. Formal Definition and Construction

Let $\mathbb{G}_0$ and $\mathbb{G}_T$ be two multiplicative cyclic groups of large prime order p , such that g is a generator of $\mathbb{G}_0$ and the discrete log problem is hard in $\mathbb{G}_0$ and $\mathbb{G}_T$. Let λ be a security parameter, and e be a bilinear map such that $e : \mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_T$. Let $\mathcal{H}_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$, $\mathcal{H}_1 : \{0, 1\}^* \rightarrow \mathbb{G}_0$, and $\mathcal{H}_2 : \mathbb{G}_T \rightarrow \{0, 1\}^*$ be three cryptographic hash functions that we model in our security

- **Revoke3**($\text{crk}_{\text{urev}}^{\text{DO}}$, umrk , $\mathcal{H}_0(\text{gid})$, st_i^{DO} , DU_{hkey} , pub_{DO} , CT): Let $\text{current_st}^{\text{DO}} := \text{st}_i^{\text{DO}}$, where st_i^{DO} denotes the tuple in st^{DO} that DO outsourced during the Revoke1 operation. Verify the signed Merkle root by computing $\prod_{2\text{Verify}}(\text{pub}_{\text{DO}}, \text{current_st}^{\text{DO}}[\text{Sig}_{\text{UL}}], \text{root}(\text{current_st}^{\text{DO}}[\text{MHT}_{\text{UL}}]))$. If the signature is valid, compute $\text{update_param} := e(g, \text{crk}_{\text{urev}}^{\text{DO}})$, $\text{CT}[\text{CS}'] := \text{CT}[\text{CS}] \cdot \text{update_param}$, and set $\text{RC1} := \text{CT}$. Under valid signature, update $\text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(2)}'] := \text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(2)}'] \cdot \text{umrk} \cdot \text{umrk}^{1/(1+\epsilon)}$ and set $\text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(3)}'] := \text{umrk}^{1/(1+\epsilon)}$ if and only if $\mathcal{H}_0(\text{gid})$ is not in $\text{current_st}^{\text{DO}}[\text{MHT}_{\text{UL}}']$.
- **Revoke4**(amrk , $\mathcal{H}_0(\text{gid})$, st_i^{AA} , st_i^{DO} , DU_{hkey} , pub_{AA} , CT , pub_{DO}): Define $\text{current_st}^{\text{DO}} := \text{st}_i^{\text{DO}}$ and $\text{current_st}^{\text{AA}} := \text{st}_i^{\text{AA}}$, where st_i^{DO} denotes the tuple in st^{DO} that DO outsourced in the Revoke1 operation, and st_i^{AA} denotes the tuple in st^{AA} that the corresponding AA outsourced in the Revoke2 operation. Verify the signature validity by computing $\prod_{2\text{Verify}}(\text{pub}_{\text{AA}}, \text{current_st}^{\text{AA}}[\text{Sig}_{\text{AL}}'], \text{current_st}^{\text{AA}}[\text{AL}'])$. If the signature is valid, for every leaf node y in $\text{CT}[\mathcal{T}']$ such that $\text{att}(y)$ belongs to $\text{current_st}^{\text{AA}}[\text{AL}']$, compute $\text{CT}[\text{D}_y'] := (\text{CT}[\text{D}_y'])^{\text{amrk}}$ and $\text{CT}[\text{T}_y'] := (\text{CT}[\text{T}_y'])^{\text{amrk}}$. Set $\text{RC2} := \text{CT}$. Verify the signed Merkle root by computing $\prod_{2\text{Verify}}(\text{pub}_{\text{DO}}, \text{current_st}^{\text{DO}}[\text{Sig}_{\text{UL}}'], \text{root}(\text{current_st}^{\text{DO}}[\text{MHT}_{\text{UL}}]))$. If the signature is valid and $\mathcal{H}_0(\text{gid})$ is not in $\text{current_st}^{\text{DO}}[\text{MHT}_{\text{UL}}']$, then compute $V := (\text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(1)}'])^{\text{amrk}}$. Next, for each attribute $\text{attr} \in \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{S}_{\text{attr}}']$ such that $\text{attr} \in \text{current_st}^{\text{AA}}[\text{AL}']$, update $\text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{attr}][\text{D}_j'] := \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{attr}][\text{D}_j'] \cdot V$, set $U := (\text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(1)}'])^{1/\mathcal{H}_0(\text{attr})}$, and then update $\text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{attr}][\text{T}_j'] := \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{attr}][\text{T}_j'] \cdot U$.
- **Transform**(CT , DU_{hkey}): For each leaf node $y \in \text{CT}[\mathcal{T}']$ such that $\text{att}(y)$ corresponds to $\text{attr} \in \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{S}_{\text{attr}}']$, where attr is indexed by j , compute $F_y := \left(\frac{e(\text{CT}[\text{D}_y'], \text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(1)}']) \cdot \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{att}(y)][\text{D}_j']}{e(\text{CT}[\text{T}_y'], \text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{att}(y)][\text{T}_j'])} \right) = e(g, g)^{(\epsilon + r \cdot a' \cdot b') \cdot q_y(0)/(1+\epsilon)}$.
For each child z of $x \in \mathcal{T}$, set $\mathcal{S}'_x := \{(\text{index}(z) | z \in \mathcal{S}_x)\}$ and compute $F_x := \prod_{z \in \mathcal{S}_x} F_z^{\Delta_{i, \mathcal{S}'_x}(0)} = e(g, g)^{(\epsilon + r \cdot a' \cdot b') \cdot q_x(i) \cdot \Delta_{i, \mathcal{S}'_x}(0)/(1+\epsilon)} = e(g, g)^{(\epsilon + r \cdot a' \cdot b') \cdot q_x(0)/(1+\epsilon)}$. Compute $\text{TC} = (\text{F}_{\text{root}(\mathcal{T})})$, $\text{sig}_{\text{TC}} := \prod_{\text{sig}_{\text{Sign}}}(\text{sk}_{\text{TEE}}, \text{TC})$. If $\text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(3)}']$ equals $\perp$, then compute $I := e(\text{CT}[\text{C}'], \text{DU}_{\text{key}}[\text{DO}_{\text{key}}'][\text{hk}_{\text{DO}(2)}'])$, else, compute $I := e(\text{CT}[\text{C}'], \text{DU}_{\text{key}}[\text{DO}_{\text{key}}'][\text{hk}_{\text{DO}(2)}']) / e(\text{CT}[\text{C}'], \text{DU}_{\text{hkey}}[\text{hk}_{\text{DO}(3)}'])$. Output $(\text{TC}, \text{sig}_{\text{TC}}, I)$. The value of TC for which $\text{DU}_{\text{hkey}}[\text{TK}_{\text{DU}}^{\text{Spu}}][\text{S}_{\text{attr}}']$ satisfies $\text{CT}[\mathcal{T}']$ yields $\text{TC} = e(g, g)^{(\epsilon + r \cdot a' \cdot b') \cdot s/(1+\epsilon)}$.
- **Decrypt**(CT , TC , I , DU_{key} , pub_{DO}): Compute the following terms.
 $\bar{F} := e(\text{CT}[\text{C}'], \text{DU}_{\text{key}}[\text{DO}_{\text{key}}'][\text{sk}_{\text{DO}}'])$.
 $B := (\bar{F} \cdot I) / \text{TC}$.
 $M' := \text{CT}[\text{CS}'] / B$.
 $W' := \mathcal{H}_1(\mathcal{H}_2(M'))$.
Compute $\prod_{2\text{Verify}}(\text{pub}_{\text{DO}}, W', \text{CT}[\text{sig}_{W'}])$ to assess the value of the witness W' , and the validity of $\text{CT}[\text{sig}_{W'}]$.
Output M' if W' is correct and $\text{CT}[\text{sig}_{W'}]$ is a valid signature. Otherwise, output $\perp$.

Fig. 4. Our decentralized and lightweight SR-ABE scheme (part II).

proof as random oracles $\mathcal{O}_{\mathcal{H}_0}$, $\mathcal{O}_{\mathcal{H}_1}$, and $\mathcal{O}_{\mathcal{H}_2}$, respectively. We assume a one-time trusted sampling of CRS via $\text{GroupGen}(\lambda)$, executed by a trusted party or through multi-party computation. We set $\text{CRS} = (\mathbb{G}_0, \mathbb{G}_T, e, p, g, \mathcal{H}_0, \mathcal{H}_1, \mathcal{H}_2)$ and assume that all participants in the framework can access it.

Let $\prod_1$ denote our construction at the ABE layer, $\prod_2$ denote our scheme at the public-key layer, and $\prod_{\text{sig}}$ denote the attestation mechanism of the TEE module with transparent execution. To ensure user unlinkability across different AAs, we leverage the Dodis-Yampolskiy PRF (DY-PRF) [41]. We model the digital signature scheme at the public-key layer, $(\prod_{2\text{Sign}}, \prod_{2\text{Verify}})$, using the Boneh-Lynn-Shacham (BLS) signature construction. Our study concentrates on the communication between DU and the TEE module for ciphertext transformation, ignoring other types of interaction with the TEE module. We present our scheme in Figs. 3 and 4, and detail the fourteen operations of our framework as follows.

- **Setup**(CRS) $\rightarrow (pk_{\text{DO}}, sk_{\text{DO}}, PP, MSK)$: DO runs the Setup operation that takes as input CRS and outputs pk_{DO} , sk_{DO} , PP and MSK .
- **SetupAA**($aaid, \mathbb{S}_{\text{AA}}$) $\rightarrow (\text{pub}_{\text{AA}}, \text{priv}_{\text{AA}}, \text{attr_set})$: Every AA wishing to register and manage attributes in the system executes the SetupAA operation. Inputs include the unique AA identity $aaid$ and $\mathbb{S}_{\text{AA}}$, which represents the set of attributes exclusive to the new AA. The operation outputs priv_{AA} , pub_{AA} , as well as attr_set . We note that every attribute has an initial version number of 0.
- **RegUser**($\text{gid}, \mathbb{S}$) $\rightarrow (\text{pub}_{\text{DU}}, \text{priv}_{\text{DU}})$: To enroll in the system, each DU runs the RegUser operation, where gid , a unique identifier, and $\mathbb{S}$, the desired attribute set, are inputs. The outputs are priv_{DU} and pub_{DU} .
- **Keygen**: We subdivide the key generation process in four operations as follows.

- 1) $Keygen1(MSK, gid, pub_{DU}) \rightarrow DO_{key}$: DO runs the Keygen1 operation that outputs DO_{key} , which DO sends to the corresponding DU.
- 2) $Keygen2(\mathcal{H}_0(gid), priv_{AA}, \mathbb{S}_{DU}) \rightarrow AA_{key}^{(DY-PRF(\mathcal{H}_0(gid)), \mathbb{S}_{DU})}$: AA runs the Keygen2 operation that outputs $AA_{key}^{(DY-PRF(\mathcal{H}_0(gid)), \mathbb{S}_{DU})}$. We require that every attribute in $\mathbb{S}_{DU}$ appears within the `attr_set` table of the specified AA. We let AA send $AA_{key}^{(DY-PRF(\mathcal{H}_0(gid)), \mathbb{S}_{DU})}$ to the corresponding DU.
- 3) $Keygen3((AA_{key}^{(DY-PRF(\mathcal{H}_0(gid)), \mathbb{S}_{DU})})_{(1, \dots, k)}) \rightarrow TK_{DU}^{\mathbb{S}_{DU}}$: DU runs the Keygen3 operation that aggregates all the $(AA_{key}^{(DY-PRF(\mathcal{H}_0(gid)), \mathbb{S}_{DU})})_{(1, \dots, k)}$ from k AAs of interest. The operation yields a transformation key $TK_{DU}^{\mathbb{S}_{DU}}$, which DU stores on premises.
- 4) $Keygen4(DO_{key}, TK_{DU}^{\mathbb{S}_{DU}}) \rightarrow DU_{key}$: DU runs the Keygen4 operation that outputs the complete user key DU_{key} . Based on DU_{key} , DU constructs and outsources DU_{hkey} to TEE-CS.
- $Encrypt(PP, M, \mathcal{T}, priv_{DO}) \rightarrow (CT)$: DO runs the encrypt operation that computes a function of the message M under the threshold tree-based access structure $\mathcal{T}$. The operation outputs CT that includes the witness W and DO's digital signature sig_W over W . DO sends CT to TEE-CS. The $Encrypt$ operation extends the foundational encryption strategy by Bethencourt et al. [5], employing Shamir's secret sharing to distribute a secret s via a node-specific secret polynomial $q(x)$ within $\mathcal{T}$.
- **Revoke**: We subdivide the indirect revocation process into four operations.
 - 1) $Revoke1(MSK, \sigma^{DO}, st_i^{DO}, UL, priv_{DO}) \rightarrow (crk_{urev}^{DO}, st_i^{DO}, umrk)$: DO runs the $Revoke1$ operation to initiate the revocation of users whose `gids` are in `UL`, as well as to generate key and ciphertext update information. DO outsources st_i^{DO} , $umrk$, and crk_{urev}^{DO} to TEE-CS.
 - 2) $Revoke2(\sigma^{AA}, attr_list, priv_{AA}, st_i^{AA}, attr_set) \rightarrow (st_i^{AA}, amrk)$: AA runs the $Revoke2$ operation to cancel the current version of an attribute, where the target attribute is specified within `attr_list` and the current version of each attribute can be found within the internal table `attr_set`. To enforce a new version number, the corresponding AA produces key and ciphertext update information for the target attribute. Finally, AA forwards st_i^{AA} and $amrk$ to TEE-CS.
 - 3) $Revoke3(crk_{urev}^{DO}, umrk, \mathcal{H}_0(gid), st_i^{DO}, DU_{hkey}, pub_{DO}, CT) \rightarrow (RC1, DU_{hkey})$: TEE-CS runs the $Revoke3$ operation to recompute the ciphertext CT following user revocation. We denote the resulting ciphertext as $RC1$. The $Revoke3$ operation also aims to update DU_{hkey} , assuming that the corresponding DU is unrevoked. If DU is revoked, TEE-CS only recomputes CT into $RC1$ and leaves DU_{hkey} unchanged. Within our security model outlined in Section III-C, we decompose $Revoke3$ into two distinct sub-operations: $Revoke3.1(st_i^{DO}, umrk, \mathcal{H}_0(gid), DU_{hkey}, pub_{DO}) \rightarrow DU_{hkey}$, which aims to update DU_{hkey} ;

$Revoke3.2(st_i^{DO}, umrk, crk_{urev}^{DO}, CT, pub_{DO}) \rightarrow RC1$, which aims to recompute CT .

- 4) $Revoke4(amrk, \mathcal{H}_0(gid), st_i^{AA}, st_i^{DO}, DU_{hkey}, pub_{AA}, CT, pub_{DO}) \rightarrow (RC2, DU_{hkey})$: TEE-CS runs the $Revoke4$ operation to recompute the ciphertext CT following attribute revocation. We denote the resulting ciphertext as $RC2$. Through the $Revoke4$ operation, TEE-CS also updates DU_{hkey} , assuming that the corresponding DU is not revoked. If DU is revoked, TEE-CS only recomputes CT into $RC2$ and leaves DU_{hkey} unchanged. In our security model, we divide $Revoke4$ in two distinct sub-operations: $Revoke4.1(st_i^{AA}, st_i^{DO}, amrk, \mathcal{H}_0(gid), DU_{hkey}, pub_{DO}, pub_{AA}) \rightarrow DU_{hkey}$, which aims to update DU_{hkey} ; $Revoke4.2(st_i^{AA}, amrk, CT, pub_{AA}) \rightarrow RC2$, which aims to recompute CT .

- $Transform(CT, DU_{hkey}) \rightarrow TC$: TEE-CS runs the $Transform$ operation to conduct ciphertext transformation. We note that the input CT is generic, encompassing the original CT and its recomputed variants $RC1$ and $RC2$. TEE-CS returns TC to DU, along with a precomputed term I .
- $Decrypt(CT, TC, I, DU_{key}, pub_{DO}) \rightarrow M'$: DU runs the $Decrypt$ operation to recover the original data. If DU fails to validate W in CT based on a locally reconstructed witness W' , then M' becomes non-compliant and the $Decrypt$ operation returns $\perp$. The $Decrypt$ operation is successful if $M' = M$.

For the purpose of correctness, we require that pub_{DU} , gid , DO_{key} , $TK_{DU}^{\mathbb{S}_{DU}}$, and $\mathbb{S}_{DU}$ belong to the same data user.

C. Security Model

In our security model, DO stands as trusted since it has knowledge of the plaintext and initializes the system at the ABE layer. TEE-CS is semi-trusted: it follows the normal protocol workflow but might launch passive or side-channel attacks to gather more data, potentially allowing it to execute the final ciphertext decryption stage. However, TEE-CS is assumed never to collude with any party, including revoked users, to undermine the security of the system. Our framework sees DUs as malicious entities, whereas AAs are considered semi-trusted: they follow the protocol but might attempt to infer useful information about DUs' attributes across different AAs.

We note that since TEE-CS (through its TEE module) operates under the transparent enclave execution model [39], the enclave execution is fully transparent to the adversary except the platform attestation mechanism that keeps the secret signing key sk_{TEE} hidden. We assume that all entities can access the public signing key pk_{TEE} to validate an attestation token from the TEE module. We rely on the transparent enclave execution model to argue whether our framework remains secure even when the adversary has unbounded capabilities to observe transformation keys (i.e., DU_{hkey}).

We formalize the IND-CPA security model of our framework as a game between a challenger $\mathcal{C}$, which controls random oracles, and a probabilistic polynomial-time (PPT) adversary

$\mathcal{A}$. Our security model captures user and attribute revocation. The following game is played between $\mathcal{C}$ and $\mathcal{A}$.

- *Setup*: $\mathcal{C}$ starts the game by running $\text{Setup}(\text{CRS})$. $\mathcal{C}$ sends $(\text{CRS}, \text{PP}, \text{pub}_{\text{DO}})$ to $\mathcal{A}$, while keeping MSK and priv_{DO} secret. In addition, $\mathcal{C}$ generates empty UL and AL , initializes an internal state tuple σ^{DO} , then adds σ^{DO} to an internal table $\text{B}_{\sigma^{\text{DO}}}$. In addition, $\mathcal{C}$ sets up an empty table RT_{DO} , which stores public state entries that support user revocation, as well as an empty table RT_{AA} , which stores public state entries that support attribute revocation.
- *Phase 1*: $\mathcal{A}$ can adaptively issue any of the following queries to $\mathcal{C}$.
 - *SetupAA(aaid, $\mathbb{S}_{\text{aaid}}$)*: $\mathcal{A}$ issues an AA creation query based on the AA's identity aaid , and the AA set of attributes $\mathbb{S}_{\text{aaid}}$. $\mathcal{C}$ computes $(\text{pub}_{\text{aaid}}, \text{priv}_{\text{aaid}})$ at the public-key layer and stores the entry $(\text{aaid}, \text{pub}_{\text{aaid}}, \text{priv}_{\text{aaid}})$ within an internal table AT . For every attribute index i of $\mathbb{S}_{\text{aaid}}$, $\mathcal{C}$ sets the initial version of each attribute, $\text{version}_{\mathbb{S}_{\text{aaid}}[i]}$, to 0, and stores a unique entry $(\mathbb{S}_{\text{aaid}}[i], \text{aaid}, \text{version}_{\mathbb{S}_{\text{aaid}}[i]})$ within an internal table SL . Moreover, $\mathcal{C}$ initializes an internal state tuple $\sigma^{\text{AA}}_{\text{aaid}}$ that it adds to an internal table of states $\text{B}_{\sigma^{\text{AA}}}$ through the entry $(\text{aaid}, \sigma^{\text{AA}}_{\text{aaid}})$. $\mathcal{C}$ returns pub_{aaid} to $\mathcal{A}$ and keeps $\text{priv}_{\text{aaid}}$ secret. If an entry with aaid is already indexed in AT , $\mathcal{C}$ outputs $\perp$.
 - *RegUser(gid, $\mathbb{S}_{\text{gid}}$)*: $\mathcal{A}$ issues a user registration query on the identity gid and the set of attributes $\mathbb{S}_{\text{gid}}$. $\mathcal{C}$ computes $(\text{pub}_{\text{gid}}, \text{priv}_{\text{gid}})$ at the public-key layer and stores the entry $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}})$ within an internal table UT . $\mathcal{C}$ returns pub_{gid} to $\mathcal{A}$ and keeps priv_{gid} secret. If an entry with gid is already indexed in UT , $\mathcal{C}$ outputs $\perp$.
 - *Keygen(gid)*: $\mathcal{A}$ issues a key query at the ABE layer on identity gid . If an entry with gid is indexed in UT , $\mathcal{C}$ fetches $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}})$ and runs $\text{DO}_{\text{key}} := \text{Keygen1}(\text{MSK}, \text{gid}, \text{pub}_{\text{gid}})$. For every index j of $\mathbb{S}_{\text{gid}}$, if there is an entry $(\mathbb{S}_{\text{gid}}[j], \text{aaid}, \text{version}_{\mathbb{S}_{\text{gid}}[j]})$ indexed by $\mathbb{S}_{\text{gid}}[j]$ in SL , $\mathcal{C}$ extracts aaid then gets $(\text{aaid}, \text{pub}_{\text{aaid}}, \text{priv}_{\text{aaid}})$ from AT , and iteratively constructs $\mathbb{S}_{\text{aaid}} \subset \mathbb{S}_{\text{gid}}$. Following the successful execution of Keygen1 , $\mathcal{C}$ runs $\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{O}_{\mathcal{H}_0}(\text{gid})), \mathbb{S}_{\text{aaid}})} := \text{Keygen2}(\mathcal{O}_{\mathcal{H}_0}(\text{gid}), \text{priv}_{\text{aaid}}, \mathbb{S}_{\text{aaid}})$ for each extracted subset $\mathbb{S}_{\text{aaid}}$. $\mathcal{C}$ computes $\text{TK}_{\text{DU}}^{\mathbb{S}_{\text{gid}}} := \text{Keygen3}((\text{AA}_{\text{key}}^{(\text{DY-PRF}(\mathcal{O}_{\mathcal{H}_0}(\text{gid})), \mathbb{S}_{\text{aaid}})})_{(1, \dots, k)})$ for the k identified $\mathbb{S}_{\text{aaid}}$, then runs $(\text{DU}_{\text{key}}, \text{DU}_{\text{hkey}}) := \text{Keygen4}(\text{DO}_{\text{key}}, \text{TK}_{\text{DU}}^{\mathbb{S}_{\text{gid}}})$, where $\text{DU}_{\text{hkey}} \subset \text{DU}_{\text{key}}$. $\mathcal{C}$ updates the entry indexed by gid within UT as $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}}, \text{DU}_{\text{hkey}}, \text{DU}_{\text{key}})$, and returns DU_{hkey} (i.e., keeping sk_{DO} secret) to $\mathcal{A}$. If there is no entry indexed by gid in UT , $\mathcal{C}$ returns $\perp$. For each $\mathbb{S}_{\text{gid}}[j]$ not in SL , $\mathcal{C}$ outputs $\perp$.
 - *CorruptDU(gid)*: $\mathcal{A}$ issues a user corruption query on identity gid . If there is an entry indexed by gid in UT , $\mathcal{C}$ checks whether $\text{gid} \in \text{UL}$. If $\text{gid} \in \text{UL}$, $\mathcal{C}$ fetches (from UT) and returns to $\mathcal{A}$ either $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}})$ if $\text{Keygen}(\text{gid})$ was not

queried, or $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}}, \text{DU}_{\text{hkey}}, \text{DU}_{\text{key}})$ if $\text{Keygen}(\text{gid})$ has been queried. If $\text{gid} \notin \text{UL}$, $\mathcal{C}$ returns either $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}})$ if $\text{Keygen}(\text{gid})$ was not queried, or $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{DU}_{\text{hkey}})$ if $\text{Keygen}(\text{gid})$ was queried. If no entry with gid is indexed in UT , $\mathcal{C}$ returns $\perp$.

- *CorruptAA(aaid)*: $\mathcal{A}$ issues an AA corruption query on identity aaid . If there is an entry indexed by aaid in AT , $\mathcal{C}$ gets $(\text{aaid}, \text{pub}_{\text{aaid}}, \text{priv}_{\text{aaid}})$ from AT , a set of entries $\{(\mathbb{S}_{\text{aaid}}[i], \text{aaid}, \text{version}_{\mathbb{S}_{\text{aaid}}[i]})\}$ from SL , and $(\text{aaid}, \sigma^{\text{AA}}_{\text{aaid}})$ from $\text{B}_{\sigma^{\text{AA}}}$. $\mathcal{C}$ returns $\text{priv}_{\text{aaid}}$, $\{(\mathbb{S}_{\text{aaid}}[i], \text{aaid}, \text{version}_{\mathbb{S}_{\text{aaid}}[i]})\}$, and $(\text{aaid}, \sigma^{\text{AA}}_{\text{aaid}})$ to $\mathcal{A}$. If no such entry exists in AT , $\mathcal{C}$ returns $\perp$.
- *RevokeDU(gid)*: $\mathcal{A}$ issues a user revocation query on identity gid . If there is an entry indexed by gid in UT , $\mathcal{C}$ sets $\text{UL} = \text{UL} \cup \{\text{gid}\}$. $\mathcal{C}$ fetches the latest σ^{DO} from $\text{B}_{\sigma^{\text{DO}}}$, derives a new value of σ^{DO} from the preceding one, and constructs MHT_{UL} by running $(\text{crk}_{\text{urev}}^{\text{DO}}, \text{st}^{\text{DO}}, \text{umrk}) := \text{Revoke1}(\text{MSK}, \sigma^{\text{DO}}, \text{RT}_{\text{DO}}, \text{UL}, \text{priv}_{\text{DO}})$. $\mathcal{C}$ inserts the updated σ^{DO} as a new entry into $\text{B}_{\sigma^{\text{DO}}}$, adds $(\text{st}^{\text{DO}}, \text{umrk})$ to RT_{DO} , then returns st^{DO} , together with a Merkle proof of gid 's inclusion into MHT_{UL} , to $\mathcal{A}$. If no entry indexed by gid exists in UT , $\mathcal{C}$ returns $\perp$.
- *RevokeATT($\mathbb{S}$)*: $\mathcal{A}$ issues an attribute revocation query on the attribute set $\mathbb{S}$. For every index i of $\mathbb{S}$, if there is an entry indexed by $\mathbb{S}[i]$ in SL , $\mathcal{C}$ gets $(\mathbb{S}[i], \text{aaid}, \text{version}_{\mathbb{S}[i]})$. Moreover, for each extracted aaid , $\mathcal{C}$ defines $\mathbb{S}_{\text{aaid}} \subset \mathbb{S}$ so that the AA with identity aaid administers $\mathbb{S}_{\text{aaid}}$. For each aaid , $\mathcal{C}$ fetches the latest entry $(\text{aaid}, \sigma^{\text{AA}}_{\text{aaid}})$ from $\text{B}_{\sigma^{\text{AA}}}$, and updates such extracted $\sigma^{\text{AA}}_{\text{aaid}}$ by computing $(\text{st}^{\text{AA}}, \text{amrk}) := \text{Revoke2}(\sigma^{\text{AA}}_{\text{aaid}}, \mathbb{S}_{\text{aaid}}, \text{priv}_{\text{AA}}, \text{RT}_{\text{AA}}, \text{SL})$, where priv_{AA} and RT_{AA} belong to the AA with identity aaid . During the Revoke2 operation, $\mathcal{C}$ increments the version number in SL for every valid entry $(\mathbb{S}[i], \text{aaid}, \text{version}_{\mathbb{S}[i]})$, transforming it into $(\mathbb{S}[i], \text{aaid}, \text{version}_{\mathbb{S}[i]} + 1)$. In addition, $\mathcal{C}$ inserts the updated $\sigma^{\text{AA}}_{\text{aaid}}$ into $\text{B}_{\sigma^{\text{AA}}}$ as a new entry $(\text{aaid}, \sigma^{\text{AA}}_{\text{aaid}})$. For each aaid , $\mathcal{C}$ stores a tuple $(\text{aaid}, \text{st}^{\text{AA}}, \text{amrk})$ as a new entry into RT_{AA} , and returns $\{(\text{aaid}, \text{st}^{\text{AA}})\}$ to $\mathcal{A}$. For each $\mathbb{S}[i]$ not indexed in SL , $\mathcal{C}$ outputs $\perp$.
- *KeyUpdate(gid, DU_{hkey})*: $\mathcal{A}$ issues a revocation key update query based on gid and DU_{hkey} . If there is an entry indexed by gid in UL , $\mathcal{C}$ outputs $\perp$. If there is no entry with gid that is indexed in UT , $\mathcal{C}$ outputs $\perp$. If there is an entry indexed by gid in UT but without DU_{hkey} and DU_{key} key components, $\mathcal{C}$ outputs $\perp$. If there is an entry with gid that is indexed in UT such that gid is not in UL and the key components DU_{hkey} and DU_{key} are available, $\mathcal{C}$ proceeds in two consecutive and related steps.
 - 1) *user revocation updates*: $\mathcal{C}$ retrieves the most recent entry $(\text{st}^{\text{DO}}, \text{umrk})$ from RT_{DO} as well as fetches $(\text{gid}, \mathbb{S}_{\text{gid}}, \text{pub}_{\text{gid}}, \text{priv}_{\text{gid}}, \text{DU}_{\text{hkey}}, \text{DU}_{\text{key}})$ from UT . $\mathcal{C}$ checks that DU_{hkey} supplied by $\mathcal{A}$ is the same as the DU_{hkey} in UT and outputs $\perp$ if not. $\mathcal{C}$ updates DU_{hkey} by running $\text{DU}_{\text{hkey}} := \text{Revoke3.1}(\text{st}^{\text{DO}}, \text{umrk}, \mathcal{O}_{\mathcal{H}_0}(\text{gid}), \text{DU}_{\text{hkey}}, \text{pub}_{\text{DO}})$.

$\mathcal{C}$ overwrites the corresponding entry in UT with the new content of DU_{hkey} . (i.e., sk_{DO} remains unchanged).

- 2) *attribute revocation updates*: for every index j of $\mathbb{S}_{\text{gid}}$, $\mathcal{C}$ obtains an entry $(\mathbb{S}_{\text{gid}}[j], \text{aaid}, \text{version}_{\mathbb{S}_{\text{gid}}[j]})$ from SL, extracts aaid, then gets the latest $(\text{aaid}, \text{st}^{\text{AA}}, \text{amrk})$ entry from RT_{AA} . $\mathcal{C}$ retrieves the latest $(\text{st}^{\text{DO}}, \text{umrk})$ entry from RT_{DO} and updates DU_{hkey} by running the operation $\text{DU}_{\text{hkey}} := \text{Revoke4.1}(\text{st}^{\text{AA}}, \text{st}^{\text{DO}}, \text{amrk}, \mathcal{O}_{\mathcal{H}_0}(\text{gid}), \text{DU}_{\text{hkey}}, \text{pub}_{\text{DO}}, \text{pub}_{\text{AA}})$.

Following the two stages above, $\mathcal{C}$ returns the updated value of DU_{hkey} to $\mathcal{A}$ or outputs $\perp$.

- *Challenge*: $\mathcal{A}$ submits the challenge user identity gid^* , two messages M_0, M_1 of equal length, and a target access structure $\mathcal{T}^*$ satisfying the following constraints:
 - 1) If $\mathcal{A}$ issued the requests $\text{RegUser}(\text{gid}^*, \mathbb{S}^*)$ and $\text{Keygen}(\text{gid}^*)$, where $\mathbb{S}^*$ satisfies $\mathcal{T}^*$: $\text{RevokeDU}(\text{gid}^*)$ must be queried immediately.
 - 2) If $\text{CorruptAA}(\text{aaid})$ queries have been made for all AAs that manage the attributes in $\mathbb{S}^*$, where $\mathbb{S}^*$ satisfies $\mathcal{T}^*$, then $\mathcal{C}$ outputs $\perp$.

The challenger $\mathcal{C}$ samples a bit $b \xleftarrow{\$} \{0, 1\}$ uniformly at random, and computes $\text{CT}^* := \text{Encrypt}(\text{PP}, \text{M}_b, \mathcal{T}^*, \text{priv}_{\text{DO}})$. Once $\mathcal{C}$ has produced CT^* , the following constraints are enforced:

- 1) If $\text{RevokeDU}(\text{gid}^*)$ was queried, $\mathcal{C}$ computes $\text{CT}^* := \text{Revoke3.2}(\text{st}_i^{\text{DO}}, \text{umrk}, \text{crk}_{\text{urev}}^{\text{DO}}, \text{CT}^*, \text{pub}_{\text{DO}})$.
- 2) If $\text{RevokeATT}(\mathbb{S}^*)$ was queried, $\mathcal{C}$ computes $\text{CT}^* := \text{Revoke4.2}(\text{st}_i^{\text{AA}}, \text{amrk}, \text{CT}^*, \text{pub}_{\text{AA}})$.

Using the above computations, $\mathcal{C}$ recomputes CT^* to incorporate all applicable ciphertext update information, then returns the resulting CT^* back to $\mathcal{A}$.

- *Phase 2*: $\mathcal{A}$ continues to issue queries to $\mathcal{C}$ as in Phase 1, with the same restrictions defined in the Challenge phase.
- *Guess*: $\mathcal{A}$ makes a guess b' for b and wins the game if $b' = b$.

We denote our server-revocable multi-authority CP-ABE framework as Π and state that the advantage of $\mathcal{A}$ in this game is defined as $\text{Adv}_{(\mathcal{A}, \Pi)}^{\text{IND-CPA}} = |\Pr[b' = b] - \frac{1}{2}|$.

Definition: Π is adaptively IND-CPA secure if any probabilistic polynomial-time adversary $\mathcal{A}$ has at most a negligible advantage $\text{Adv}_{(\mathcal{A}, \Pi)}^{\text{IND-CPA}}$ within the security game defined above in the security parameter λ (in CRS).

Remark: To cover transformation key exposure attacks in our IND-CPA security model, we allow $\mathcal{A}$ to access DU 's transformation key (i.e., DU_{hkey}) through leakage or corruption.

IV. SECURITY ANALYSIS

This section presents the formal security proof of our construction and analyzes the collusion resistance property of our scheme.

A. Formal Security Proof

We establish the security of our framework in the one-user setting [26], [29]. Different from the original definition, which

considers a single target user and classifies adversaries into two categories based on the challenge time t^* , we adapt the one-user setting to our state-triggered scenario.

In our work, the one-user setting likewise presumes a unique target user whose attributes satisfy the challenge access structure. However, if this target user is revoked, the adversary may obtain both the target user's secret key and transformation key through corruption. In addition, when the target user is not revoked, the adversary is instead limited to learning only the target user's transformation key, reflecting the inherent leakage that is intrinsic to our use of the transparent enclave execution model within the TEE module. Furthermore, the original one-user setting was formulated for selective security, whereas our scheme extends its definition towards fully adaptive (IND-CPA) security.

The following two conditions hold simultaneously in the one-user setting within our security model:

- 1) There is a single user in the system (also known as the target user), whose attributes satisfy the challenge access structure $\mathcal{T}^*$ in CT^* .
- 2) The adversary $\mathcal{A}$ belongs to one of the following two mutually exclusive types:
 - 1) *Type 1*: $\mathcal{A}$ is allowed to corrupt any revoked DU and obtain $\text{sk}_{\text{DO}}, \text{hk}_{\text{DO}(1)}, \text{hk}_{\text{DO}(2)}, \text{hk}_{\text{DO}(3)}$, and $\text{TK}_{\text{DU}}^{\mathbb{S}_{\text{gid}}}$.
 - 2) *Type 2*: $\mathcal{A}$ is allowed to obtain the values $\text{hk}_{\text{DO}(1)}, \text{hk}_{\text{DO}(2)}, \text{hk}_{\text{DO}(3)}$, and $\text{TK}_{\text{DU}}^{\mathbb{S}_{\text{gid}}}$ from any unrevoked and non corrupt DU, including the value of $\text{TK}_{\text{DU}}^{\mathbb{S}^*}$ that satisfies $\mathcal{T}^*$.

We leave it for future work to prove the security of our scheme in the general multi-user setting [40], where both types of adversaries act concurrently against multiple target users. We note that our construction diverges substantially from the multi-user settings of Leixiao Cheng and Fei Meng [40], which is based on time-driven revocation. Accordingly, addressing the multi-user scenario within our framework remains an open challenge that we defer to future work. We base our construction on the DBDH hardness assumption and argue whether any probabilistic polynomial-time (PPT) adversary, even when granted access to transformation keys, can successfully undermine the security of our scheme.

Theorem: If the DBDH hardness assumption holds, then no PPT type 1 or type 2 adversary can win the adaptive IND-CPA security game in subsection III-C with a non-negligible advantage.

Proof: Suppose there exists a PPT adversary $\mathcal{A}$ that breaks our multi-authority SR-ABE scheme by winning the IND-CPA security game with non-negligible advantage ϵ . Then we can construct a PPT adversary $\mathcal{B}$ that, using $\mathcal{A}$ as a subroutine, solves the DBDH problem with advantage at least $\frac{\epsilon}{2}$. Throughout our proof, we demonstrate how to employ $\mathcal{A}$ to construct adversary $\mathcal{B}$ and reason about the existence of such adversary $\mathcal{B}$, which would contradict the hardness of the DBDH assumption and thus break the security of our scheme. Therefore, if the DBDH problem is indeed hard, then adversary $\mathcal{A}$ cannot exist.

Let $\mathcal{M}$ be a PPT machine that generates a structured common reference string (CRS) by taking as input a security parameter

λ . $\mathcal{M}$ gives CRS to all the parties. The DBDH challenger flips a fair coin $\mu := \{0, 1\}$. If $\mu = 0$, the DBDH challenger lets $(g, A, B, C) := (g, g^a, g^b, g^c, Z := e(g, g)^{a \cdot b \cdot c})$; if $\mu = 1$, the DBDH challenger lets $(g, A, B, C) := (g, g^a, g^b, g^c, Z := e(g, g)^z)$, where a, b, c, z are chosen randomly from $\mathbb{Z}_p$. The DBDH challenger then gives CRS and (g, A, B, C, Z) to $\mathcal{B}$.

$\mathcal{B}$ plays the role of a simulator in the DBDH game with $\mathcal{A} := \{\mathcal{A}_1, \mathcal{A}_2\}$, where $\mathcal{A}_1$ stands as type 1 adversary and $\mathcal{A}_2$ stands as type 2 adversary. $\mathcal{B}$ chooses the same CRS as the one given by $\mathcal{M}$ and sets $a := \Sigma d_k$, $b := \frac{\Sigma u_k}{\Sigma v_k}$, and $c := s_0$, where $d_1 \dots d_k, u_1 \dots u_k, v_1 \dots v_k, s_0 \in \mathbb{Z}_p$. The goal of $\mathcal{B}$ is to output 0 if $Z = e(g, g)^{a \cdot b \cdot c}$ and 1 otherwise. In summary, the security reduction demonstrates that adversary $\mathcal{B}$ cannot violate the DBDH hardness assumption. Therefore, adversary $\mathcal{A}$ likewise fails to compromise our construction, which confirms the security of our scheme under the adaptive IND-CPA attack model. We provide the full proof of our construction in the appendix of this paper.

B. Collusion Resistance

This work focuses on four types of collusion attacks. First, our scheme must prevent two illegitimate users (whether unauthorized or revoked) from combining their secret keys to gain higher decryption rights. Second, our scheme must guarantee that, even if all compromised AAs collude and pool the attribute authority secret keys they have previously issued for a given user, they are still unable to alter all attribute-related key components within that user's transformation key. Third, our scheme must prevent compromised AAs from colluding with revoked users to illegally elevate those users' privileges. Fourth, our scheme must ensure that unrevoked users cannot collude with revoked users. Below, we describe the various mechanisms that work together in our framework to thwart the collusion attacks presented above.

- 1) To mitigate the first type of collusion attacks, we incorporate the hash of a user's gid as an exponent into both the secret key and the transformation key. Under the assumption that the discrete logarithm problem is hard, this makes it computationally infeasible for illegitimate users to successfully collude.
- 2) To counter the second type of collusion attacks, we require that at least one AA remain honest throughout the entire lifetime of the system. As specified within our security game, if all AAs participating in the generation of attribute secret keys for a given user become corrupted, then the challenger outputs $\perp$.
- 3) To address the third type of collusion attacks, our framework divides both revocation and key generation responsibilities between DO and AA. In our revocation protocol, DO, which is trusted, handles the generation of key and ciphertext update information required for user revocation, whereas AA is restricted to managing attribute revocation only. This separation ensures that AA cannot access any confidential information related to user revocation updates.
- 4) For the fourth type of collusion attacks, our framework prevents unrevoked users from colluding with revoked

users by embedding the hash of each user's gid as an exponent within the key update information. As a result, every user obtains a unique update component for their transformation key. Provided that the discrete logarithm problem remains computationally hard, no unrevoked user can derive the master recomputation key (i.e., umrk) from their transformation key.

V. PERFORMANCE EVALUATION

This section provides a concise evaluation of our proposed framework (SR-MACP-ABE) from both theoretical and empirical perspectives.

A. Theoretical Evaluation

In Table II, we present a comparison between our construction and several state-of-the-art schemes from the literature, specifically [12], [16], [24], [18], and [42]. The comparison examines seven key dimensions: the type of revocation mechanism employed (RT), distinguishing between direct and indirect revocation; the trust assumption on the cloud server (semi-trusted vs. untrusted); support for time-based revocation (TBR); whether security is proven under a selective or adaptive attack model; whether the scheme offers decryption key exposure resistance (DKER); the asymptotic size of key updates (KUS); and the asymptotic size of the user's secret key (UKS). In addition, Table II employs the following notations: N denotes the number of users; R is the number of revoked users; m denotes the size of the attribute set tied to the ciphertext; k is the size of the attribute set bound to the user key at the ABE layer; ψ denotes the lazy revocation factor, defined as the fraction of non-revoked users that concurrently request revocation updates, with $\psi \in [0, 1]$; TEE stands for trusted execution environment; and “—” indicates that a given entry is not applicable. We note that $O(R \cdot \log(N/R))$ is practical as long as R remains small, but its performance deteriorates significantly when R becomes large [8].

We evaluate and contrast the computational overhead incurred at various stages of the ABE pipeline for our proposed scheme and several existing works [25], [43], [44]. Yang et al. [25] designed a generic framework that relies on a trusted KGC and converts a time-based revocable CP-ABE scheme into a time-based SR-ABE scheme (based on CP-ABE), while ensuring verifiability against an unpredictable cloud server responsible for ciphertext transformation. Han et al. [43] introduced a CP-ABE scheme that assumes a fully trusted authority and supports user (direct) revocation via the binary tree technique, partially hidden access policies, and offers white-box traceability. Chen et al. [44] proposed a revocable CP-ABE construction with a trusted authority center that leverages ciphertext delegation [16] to achieve revocation.

Tables III and IV present our findings with respect to theoretical efficiency performance. In the following, we describe the notations employed in both Tables III and IV: $\mathbb{U}$ denotes the set of data users; l denotes the size of the attribute set associated with the ciphertext; k denotes the size of the attribute set associated with the user key at the ABE layer; — denotes not-applicable;

TABLE II
FEATURE-WISE COMPARISON WITH EXISTING RABE CONSTRUCTIONS

Scheme	RT	Server-aided	TBR	Security	DKER	KUS	UKS
[12]	direct/indirect	–	✓	selective	✗	$O(R \cdot \log(N/R))$	$O(m \cdot \log(N))$
[16]	indirect	semi-trusted	✓	adaptive	✗	$O(R \cdot \log(N/R))$	$O(m \cdot \log(N))$ & $O(k \cdot \log(N))$
[24]	indirect	untrusted	✓	selective	✗	$O(R \cdot \log(N/R))$	$O(1)$
[29]	indirect	untrusted	✓	selective	✓	$O(R \cdot \log(N/R))$	$O(1)$
[18]	direct + TEE	untrusted	✗	selective	✓	–	$O(1)$
[42]	indirect	–	✓	adaptive	✓	$O(R \cdot \log(N/R))$	$O(1)$
[28]	indirect	untrusted	✓	selective	✓	$O(R \cdot \log(N/R))$	$O(k \cdot \log(N))$
Our work	indirect + TEE	semi-trusted	✗	adaptive	✓	$O(\psi \cdot (N - R))$	$O(1)$

TABLE III
THEORETICAL COMPUTATION COMPARISON WITH STATE-OF-THE-ART REVOCABLE ABE FRAMEWORKS

Scheme	Setup	KeyGen	Encrypt	Transform	Decrypt
[25]	$2C_p + 5C_e$	$(7 + 19k + 3x + 2r + 2d)C_e$	$(8 + 6l + 6lc)C_e$	$7C_p + 6nC_e$	$C_p + 2C_e$
[43]	$C_p + (2 U - 2)C_e$	$(5 + 2k)C_e$	$(3 + 4l + r)C_e$	–	$(1 + 3n + d)C_p + nC_e$
[44]	$5C_p + 2C_e$	$(4 + 18k)C_e$	$(12 + 12l + 18lc)C_e$	–	$6C_p + (2 + 6n)C_e$
Our work	$C_p + 4C_e$	$(3 + 5k)C_e$	$(1 + 2l)C_e$	$(2n + 1)C_p$	C_p

TABLE IV
STORAGE COST COMPARISON WITH STATE-OF-THE-ART REVOCABLE ABE FRAMEWORKS

Scheme	Ciphertext Size	User Key Size
[25]	$(5 + 3l) G_0 $	$(8 + 5k + x + 2r + 2d) G_0 + 2 Z_p $
[43]	$ G_T + (2 + 3l) G_0 $	$(4 + k) G_0 + Z_p $
[44]	$2 G_T + 6l G_0 + 7 G_0 $	$(6 + 3k) G_0 $
Our work	$ G_T + (2 + 2l) G_0 $	$(3 + 2k) G_0 $

n denotes the number of attributes in the decryption key that satisfies the access policy; c denotes the number of columns in the share generating matrix; r denotes the length of cover(R) that represents the minimum cover set associated with the revocation list R within the binary user tree; x denotes the length of path(u) with u being a leaf node within the user binary tree; d denotes $\text{path}(u) \cap \text{cover}(R)$; C_e denotes the cost of an exponentiation; C_p denotes the cost of a pairing; $|G_0|$ denotes the size of an element in G_0 ; $|G_T|$ denotes the size of an element in G_T ; $|Z_p|$ denotes the size of an element in Z_p . We assume that it takes a constant unit of time ($\mathcal{O}(1)$) to perform any hash computation.

B. Empirical Evaluation

We implement our scheme in Python 3.10.12 using the charm-crypto library version 0.50. We base our experiments on the SS512 symmetric elliptic curve with a 512-bit size of the prime base field. Our framework was implemented on Ubuntu 22.04.5 LTS 64-bit, running on an HP Omen 16 equipped with a 13th-generation Intel i7-13650HX processor and 16 GB of RAM.

We vary the sizes of both the attribute set and the revoked user set from 10 to 100, conduct six trials for each size, and derive the final results by averaging the measurements corresponding to each set. Because the transparent enclave execution model [39] is best interpreted as an attack model (i.e., a deliberately weakened abstraction of a TEE) rather than a realistic system architecture, and since no TEE provider would intentionally ship hardware that fully reveals its internal state, we restrict the transparent enclave execution settings to our security model and exclude any concrete implementation of such a design from the scope of this work.

During our experiments, we leveraged three AAs and a single non-revoked target user. To streamline the analysis of the collected data, we aggregate the Keygen1, Keygen2, Keygen3, and Keygen4 measurements into a unified Keygen operation. A similar approach is used for user revocation, combining Revoke1 and Revoke3, as well as for attribute revocation, merging Revoke2 and Revoke4.

Figs. 5 and 6 empirically compare our work with Kan Yang and Xiaohua Jia (KANYANG-XIAOHUAJIA) [45] as well as Yannis Rouselakis and Brent Waters (ROUSELAKIS-WATERS) [46] MA-ABE schemes. In addition, the publicly available charm-crypto implementation of Xiaohua Jia (KANYANG-XIAOHUAJIA) [45], as well as the ROUSELAKIS-WATERS (ROUSELAKIS-WATERS) [46] implementation provided by Doreen Riepel and Hoeteck Wee [47], both rely on type I (i.e., symmetric) pairings.

The scheme by Kan Yang and Xiaohua Jia (KANYANG-XIAOHUAJIA) [45] assumes a fully trusted central authority and supports attribute revocation via a single piece of update affecting relevant keys and ciphertexts. The well-known construction of Yannis Rouselakis and Brent Waters (ROUSELAKIS-WATERS) [46], based on the first truly decentralized MA-ABE construction by Allison Lewko and Brent Waters [7], supports repeated and arbitrary string attributes within the access policy, yet does not address the revocation of users or attributes.

We found that our SR-MACP-ABE scheme improves efficiency over KANYANG-XIAOHUAJIA [45] and ROUSELAKIS-WATERS [46], achieving faster system initialization (Fig. 5(a)), key generation (Fig. 5(b)), payload encryption (Fig. 5(c)), decryption (Fig. 5(d)), and attribute revocation (Fig. 5(g)), while keeping the user registration

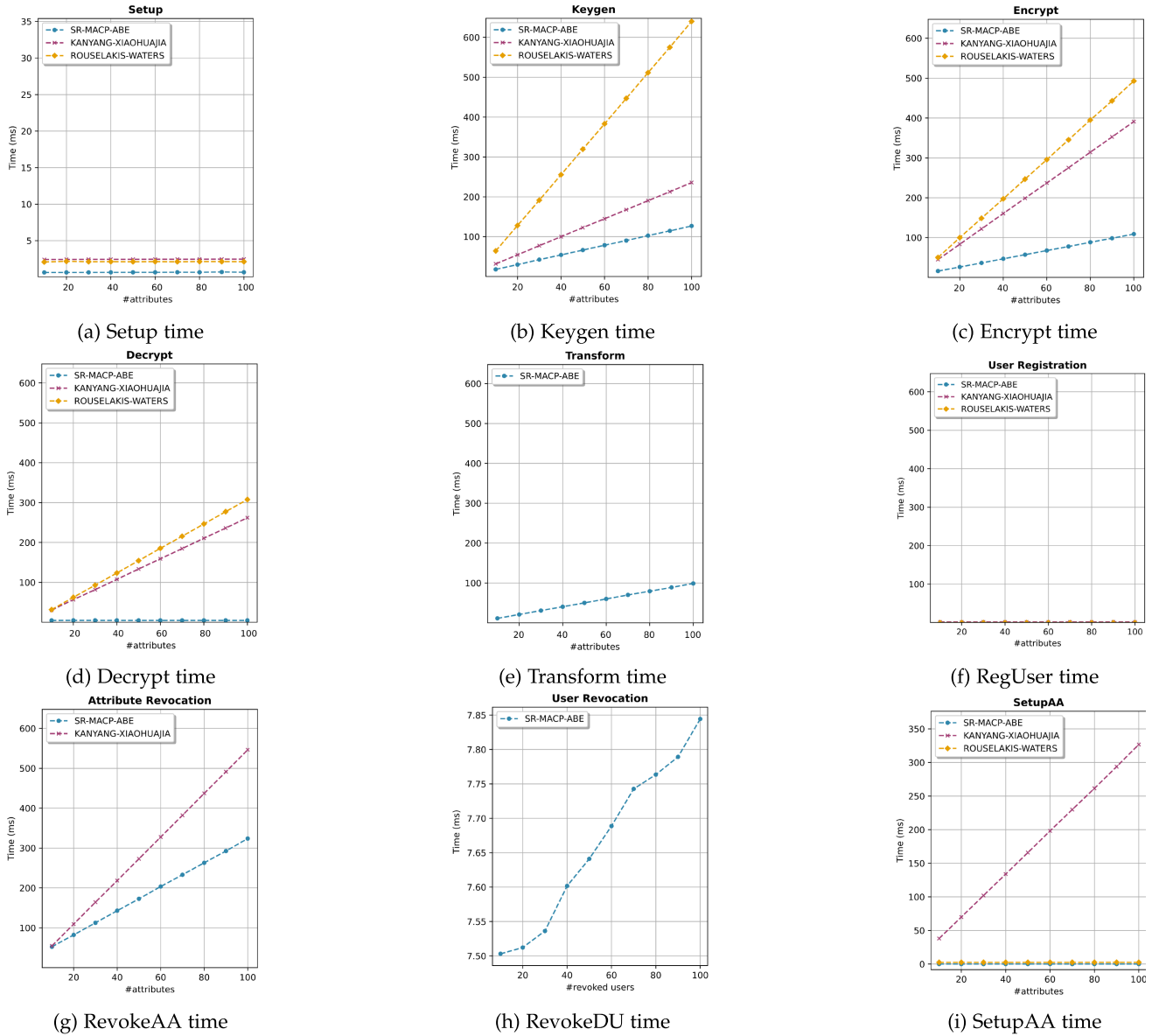


Fig. 5. Timing evaluation of the core operations in our scheme

(Fig. 5(f)) times similar. These gains come from lightweight system initialization and key generation processes, simple threshold-tree access policies instead of LSSS matrices, and outsourcing the intensive part of decryption to a semi-trusted cloud via the Transform operation (Fig. 5(e)). Attribute revocation is efficient because a single new exponent updates all key components linked to revoked attributes, aided by attribute versioning. As a result, the authority setup (Fig. 5(i)) in our construction incurs low cost, almost matched by ROUSELAKIS-WATERS [46], while being significantly more efficient than KANYANG-XIAOHUAIJIA [45]. The user revocation operation (Fig. 5(h)), evaluated only for our scheme since it is the only one offering this capability, remains efficient, taking 7.84 ms for 100 users.

According to Fig. 6(a), ciphertext sizes in our scheme fall between the two baselines, with a modest overhead caused by

extra components needed for attribute revocation and verifiable outsourced decryption. Fig. 6(b) illustrates that the size of the Merkle tree used for user revocation grows linearly with the number of revoked users. This is practically acceptable since common revocation schemes based on the binary user tree also exhibit, on average, linear-time complexity when many users are revoked. However, capping the number of users to maintain a small binary user tree inherently constrains the scalability of the system.

To our knowledge, SR-MACP-ABE is the first server-aided revocable ABE scheme that eliminates time-based assumptions for both user and attribute revocation, supports a super-polynomial number of users and attributes, and delegates revocation updates to a semi-honest verifiable cloud without incurring previous trust or scalability issues. Despite moderately larger ciphertext and Merkle tree sizes, its design remains practical and

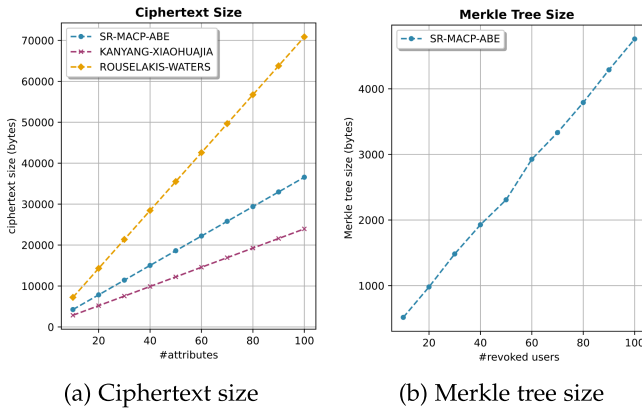


Fig. 6. Assessment of ciphertext and Merkle tree sizes

provably secure for fine-grained and dynamic access control in mobile cloud computing.

VI. CONCLUSION AND FUTURE WORK

In this paper, we address the current limitations of server-aided revocable ABE schemes for mobile cloud computing in multi-authority ABE settings. We eliminate the full trust in a third party central authority by decentralizing the generation of user long-term keys between the data owner, selected attribute authorities, and the data user. We mitigate the issue of network asynchrony in current time-based revocation systems through the implementation of state-triggered indirect revocation updates for users and attributes. We outsource both the verifiable transformation of ciphertexts and the revocation of users and attributes to a semi-trusted cloud server equipped with a TEE that operates under the theoretical assumptions of the transparent enclave execution model. We formulate and capture transformation key exposure attacks, as well as establish the framework's fully adaptive (IND-CPA) security based on the DBDH assumption in the random oracle model. In future work, we will target outsourcing attribute-related overhead away from the encryption phase, optimizing both the construction and storage of the Merkle tree used for user revocation, and establishing security within a comprehensive multi-user threat model.

REFERENCES

- [1] Y. Wu et al., "Efficient server-aided secure two-party computation in heterogeneous mobile cloud computing," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 6, pp. 2820–2834, Nov./Dec. 2021.
- [2] M. Armstrong, "Chart: Global data creation is about to explode," *Statista*, vol. 4, Apr. 16, 2019. Accessed: Sep. 18, 2025. [Online]. Available: <https://www.statista.com/chart/17727/global-data-creation-forecasts/>
- [3] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. 24th Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, Berlin, Germany, 2005, pp. 457–473.
- [4] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
- [5] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 321–334.
- [6] M. Chase, "Multi-authority attribute based encryption," in *Proc. 4th Theory Cryptogr. Conf.*, 2007, vol. 4392, pp. 515–534.
- [7] A. Lewko and B. Waters, "Decentralizing attribute-based encryption," in *Proc. Adv. Cryptol.*, K. G. Paterson, Ed., Berlin, Germany, 2011, pp. 568–588.
- [8] A. Boldyreva, V. Goyal, and V. Kumar, "Identity-based encryption with efficient revocation," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, New York, NY, USA, 2008, pp. 417–426.
- [9] J. Staddon, P. Golle, M. Gagné, and P. Rasmussen, "A content-driven access control system," in *Proc. 7th Symp. Identity Trust Internet*, New York, NY, USA, 2008, pp. 26–35.
- [10] M. Pirretti, P. Traynor, P. McDaniel, and B. Waters, "Secure attribute-based systems," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, New York, NY, USA, 2006, pp. 99–112.
- [11] S. Yu, C. Wang, K. Ren, and W. Lou, "Attribute based data sharing with attribute revocation," in *Proc. 5th ACM Symp. Inf., Comput. Commun. Secur.*, New York, NY, USA, 2010, pp. 261–270.
- [12] N. Attrapadung and H. Imai, "Attribute-based encryption supporting direct/indirect revocation modes," in *Proc. Cryptogr. Coding*, M. G. Parker, Ed., Berlin, Germany, 2009, pp. 278–300.
- [13] M. Zhang, "New model and construction of ABE: Achieving key resilient-leakage and attribute direct-revocation," in *Proc. Inf. Secur. Privacy*, W. Susilo and Y. Mu, Eds., Cham, Switzerland: Springer, 2014, pp. 192–208.
- [14] Z. Liu and D. S. Wong, "Practical Ciphertext-policy attribute-based encryption: Traitor tracing, revocation, and large universe," in *Proc. Appl. Cryptogr. Netw. Secur.*, T. Malkin, V. Kolesnikov, A. B. Lewko, and M. Polychronakis, Eds., Cham, Switzerland, 2015, pp. 127–146.
- [15] K. Yamada, N. Attrapadung, K. Emura, G. Hanaoka, and K. Tanaka, "Generic constructions for fully secure revocable attribute-based encryption," in *Proc. Comput. Secur.*, S. N. Foley, D. Gollmann, and E. Sneekenes, Eds., Cham, Switzerland, 2017, pp. 532–551.
- [16] A. Sahai, H. Seyalioglu, and B. Waters, "Dynamic credentials and ciphertext delegation for attribute-based encryption," in *Proc. Adv. Cryptol.*, R. Safavi-Naini and R. Canetti, Eds., Berlin, Germany, Springer, 2012, pp. 199–217.
- [17] X. Xie, H. Ma, J. Li, and X. Chen, "New ciphertext-policy attribute-based access control with efficient revocation," in *Proc. Inf. Commun. Technol.*, K. Mustofa, E. J. Neuhold, A. M. Tjoa, E. Weippl, and I. You, Eds., Berlin, Germany, Springer, 2013, pp. 373–382.
- [18] X. Li et al., "Make revocation cheaper: Hardware-based revocable attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, Los Alamitos, CA, USA, May 2024, pp. 3109–3127.
- [19] K. E. Fu, "Group sharing and random access in cryptographic storage file systems," Master Eng. thesis, Massachusetts Inst. Technol., Cambridge, MA, USA, 1999, pp. 79–83. [Online]. Available: <https://hdl.handle.net/1721.1/80534>
- [20] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang, and K. Fu, "Plutus: Scalable secure file sharing on untrusted storage," in *Proc. 2nd USENIX Conf. File Storage Technol.*, 2003, pp. 29–42.
- [21] S. Yu, C. Wang, K. Ren, and W. Lou, "Achieving secure, scalable, and fine-grained data access control in cloud computing," in *Proc. 29th Conf. Inf. Commun.*, 2010, pp. 534–542.
- [22] K. Yang, X. Jia, and K. Ren, "Attribute-based fine-grained access control with efficient revocation in cloud storage systems," in *Proc. 8th ACM SIGSAC Symp. Inf., Comput. Commun. Secur.*, 2013, pp. 523–528.
- [23] B. Qin, R. H. Deng, Y. Li, and S. Liu, "Server-aided revocable identity-based encryption," in *Proc. Comput. Secur.*, G.P.Y. Pernul, A. Ryan, and E. Weippl, Eds., Cham, Switzerland, 2015, pp. 286–304.
- [24] H. Cui, R. H. Deng, Y. Li, and B. Qin, "Server-aided revocable attribute-based encryption," in *Proc. Comput. Secur.*, I. Askoxylakis, S. Ioannidis, S. Katsikas, and C. Meadows, Eds., Cham, Switzerland, 2016, pp. 570–587.
- [25] F. Yang et al., "Generic constructions of server-aided revocable abe with verifiable transformation," in *Proc. Appl. Cryptogr. Netw. Secur. Workshops, Satell. Workshops, ADSC, AIBlock, AIHWS, AIoT, CIMSS, Cloud, SCI, SecMT, SiMLA*, Berlin, Heidelberg, Jun. 2023, pp. 459–479.
- [26] B. Qin, Q. Zhao, D. Zheng, and H. Cui, "(Dual) server-aided revocable attribute-based encryption with decryption key exposure resistance," *Inf. Sci.*, vol. 490, pp. 74–92, 2019.
- [27] J. H. Seo and K. Emura, "Revocable identity-based encryption revisited: Security model and construction," in *Proc. Public-Key Cryptogr.*, K. Kurosawa and G. Hanaoka, Eds., Berlin, Germany, 2013, pp. 216–234.
- [28] S. Xu, G. Yang, and Y. Mu, "Revocable attribute-based encryption with decryption key exposure resistance and ciphertext delegation," *Inf. Sci.*, vol. 479, pp. 116–134, 2019.
- [29] B. Qin, Q. Zhao, D. Zheng, and H. Cui, "Server-aided revocable attribute-based encryption resilient to decryption key exposure," in *Proc. Cryptol. Netw. Secur.*, S. Capkun and S. S. M. Chow, Eds., Cham, Switzerland, 2018, pp. 504–514.

- [30] Z. Guo et al., "UR-CP-ABE: CP-ABE with flexible construction mechanism and efficient user revocation capability for access control in the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 5989–6004, May/Jun. 2026.
- [31] H. Yin, J. Li, Y. Zhu, L. Zhang, and B. Qiao, "A traceable CP-ABE scheme supporting dynamic revocation and efficient decryption for medical data sharing," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 53610–53622, Dec. 2025.
- [32] F. Meng and L. Cheng, "TSR-ABE: Traceable and server-aided revocable ciphertext-policy attribute-based encryption under static assumptions," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 955–967, 2025.
- [33] J. Li, E. Zhang, Y. Zhang, J. Ning, and J. Shen, "A traceable and revocable ciphertext policy attribute-based encryption with policy authentication," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 2, pp. 2962–2973, Mar./Apr. 2026.
- [34] J. Sun et al., "Privacy-preserving fine-grained data sharing with dynamic service for the cloud-edge IoT," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1329–1346, Mar./Apr. 2025.
- [35] J. Wei, X. Chen, J. Wang, X. Huang, and W. Susilo, "Securing fine-grained data sharing and erasure in outsourced storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 552–566, Feb. 2023.
- [36] J. Wei, X. Chen, J. Wang, X. Hu, and J. Ma, "Enabling (end-to-end) encrypted cloud emails with practical forward secrecy," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2318–2332, Jul./Aug. 2022.
- [37] X. Chen, J. Li, X. Huang, J. Li, Y. Xiang, and D. S. Wong, "Secure outsourced attribute-based signatures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 12, pp. 3285–3294, Dec. 2014.
- [38] J. Li, X. Huang, J. Li, X. Chen, and Y. Xiang, "Securely outsourcing attribute-based encryption with checkability," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 8, pp. 2201–2210, Aug. 2014.
- [39] F. Tramèr, F. Zhang, H. Lin, J.-P. Hubaux, A. Juels, and E. Shi, "Sealed-glass proofs: Using transparent enclaves to prove and sell knowledge," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2017, pp. 19–34.
- [40] L. Cheng and F. Meng, "Server-aided revocable attribute-based encryption revised: Multi-user setting and fully secure," in *Proc. Comput. Secur.*, E.H. Bertino Shulman and M. Waidner, Eds., Cham, Switzerland, 2021, pp. 192–212.
- [41] Y. Dodis and A. Yampolskiy, "A verifiable random function with short proofs and keys," in *Proc. 8th Int. Conf. Theory Pract. Public Key Cryptogr.*, Berlin, Germany, 2005, pp. 416–431.
- [42] S. Xu, J. Ning, J. Ma, G. Xu, J. Yuan, and R. H. Deng, "Revocable policy-based chameleon hash," in *Proc. Comput. Secur. – ESORICS 2021: 26th Eur. Symp. Res. Comput. Secur.*, Darmstadt, Germany, Oct. 2021, pp. 327–347.
- [43] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 316–327, Jan./Feb. 2022.
- [44] S. Chen, J. Li, Y. Zhang, and J. Han, "Efficient revocable attribute-based encryption with verifiable data integrity," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10441–10451, Mar. 2024.
- [45] K. Yang and X. Jia, "Expressive, efficient, and revocable data access control for multi-authority cloud storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 7, pp. 1735–1744, Jul. 2014.
- [46] Y. Rouselakis and B. Waters, "Efficient statically-secure large-universe multi-authority attribute-based encryption," in *Proc. Financial Cryptogr. Data Secur.*, R. Böhme and T. Okamoto, Eds., Berlin, Germany, 2015, pp. 315–332.
- [47] D. Riepel and H. Wee, "FABEO: Fast attribute-based encryption with optimal security," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2022, pp. 2491–2504.



Arthur Sandor Voundi Koe (Member, IEEE) received the BE degree in network and computer maintenance from the African Institute of Computer Science, Cameroon, in 2010, the BS degree in fundamental computer science from the University of Yaoundé I, Cameroon, in 2011, the MS degree in computer and application technology from Hunan University, China, in 2015, and the PhD degree in computer science and application technology from Hunan University in 2020. From 2020 to 2023, he was a postdoctoral researcher with Guangzhou University.

He is currently a lecturer with Xidian University. His research interests include applied cryptography, security and privacy in cloud computing, and blockchain security.



Jianghong Wei received the PhD degree in cryptography from Zhengzhou Information Science and Technology Institute, in 2016. He is currently an associate professor with the State Key Laboratory of Mathematical Engineering and Advanced Computing. He has authored or coauthored more than 100 research papers in refereed international journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Parallel and Distributed Systems*, *USENIX Security*, *ASIACRYPT*, and *ESORICS*. His research interests include applied cryptography, privacy preservation, and machine learning security.



Xiaofeng Chen (Fellow, IEEE) received the BS and MS degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the PhD degree in Cryptography from Xidian University, in 2003. He is currently a full professor Xidian University. He has authored or coauthored more than 200 research papers in refereed international conferences and journals. His research interests include applied cryptography and cloud computing security. His work has been cited more than 19000 times on Google Scholar. He is a member of the editorial board of

IEEE Transactions on Dependable and Secure Computing, *IEEE Transactions on Knowledge and Data Engineering*, and *International Journal of Foundations of Computer Science*. He has served as the program/general chair or program committee member in more than 30 international conferences.